

TẬP ĐOÀN CÔNG NGHIỆP - VIỄN THÔNG QUÂN ĐỘI

BÁO CÁO MINI-PROJECT

Nghiên cứu Feature Flag và xây dựng hệ thống quản lý phát hành tính năng an toàn

Bùi Đình Bảo

dinhbao.itech@gmail.com - 0975 742 607

Chương trình Viettel Digital Talent 2026

Lĩnh vực: Software Engineer

Mentor:	Vũ Tuấn Đạt
Đơn vị:	Viettel Networks

Lời cảm ơn

Trước hết, em xin gửi lời cảm ơn chân thành đến Ban Tổ chức chương trình Viettel Digital Talent 2026 đã tạo điều kiện để em được tham gia học tập, rèn luyện và phát triển trong một môi trường chuyên nghiệp.

Em xin cảm ơn các anh mentor đã trực tiếp giảng dạy và chia sẻ kiến thức trong các buổi training của chương trình. Mỗi buổi đào tạo với một chủ đề và góc nhìn khác nhau đã mang lại cho em nhiều kiến thức, kinh nghiệm thực tế và những định hướng có giá trị trong quá trình học tập và hoàn thiện bản thân.

Em cũng xin cảm ơn mentor phụ trách Mini Project cùng Ban Tổ chức đã cung cấp định hướng, yêu cầu và những góp ý cần thiết trong quá trình thực hiện. Sự hỗ trợ của các anh đã giúp em xác định rõ mục tiêu, chủ động hơn trong quá trình nghiên cứu và hoàn thành sản phẩm.

Bên cạnh đó, em xin cảm ơn các bạn học viên trong chương trình đã cùng trao đổi, chia sẻ và hỗ trợ nhau trong suốt quá trình tham gia Viettel Digital Talent 2026.

Mặc dù đã dành nhiều thời gian và nỗ lực để hoàn thành báo cáo và sản phẩm, do kinh nghiệm còn hạn chế, nội dung thực hiện khó tránh khỏi những thiếu sót. Em rất mong nhận được những ý kiến đóng góp từ Ban Tổ chức và các anh/chị mentor để có thể tiếp tục cải thiện và hoàn thiện bản thân.

Em xin chân thành cảm ơn!

TP. Hồ Chí Minh, tháng 7 năm 2026

Học viên thực hiện

Bùi Đình Bảo

Lời mở đầu

Trong giai đoạn đầu của ngành công nghiệp phần mềm, các hệ thống thường được phát triển và phát hành theo những phiên bản lớn, với chu kỳ kéo dài từ vài tuần đến vài tháng. Mỗi lần phát hành thường bao gồm nhiều thay đổi được đóng gói và triển khai cùng nhau. Trong mô hình này, việc đưa mã nguồn lên môi trường production và việc cung cấp tính năng cho người dùng gần như là một hoạt động thống nhất: khi phiên bản mới được triển khai, các tính năng bên trong phiên bản đó cũng đồng thời được phát hành.

Cách tiếp cận trên tương đối phù hợp với những hệ thống có tốc độ thay đổi thấp. Tuy nhiên, khi quy mô phần mềm, số lượng người dùng và tần suất cập nhật ngày càng tăng, việc phát hành nhiều thay đổi cùng lúc làm gia tăng đáng kể rủi ro vận hành. Một lỗi trong tính năng mới có thể ảnh hưởng đến toàn bộ người dùng, trong khi quá trình khắc phục thường yêu cầu rollback phiên bản hoặc thực hiện một lần triển khai mới. Bên cạnh đó, việc duy trì các nhánh phát triển tồn tại trong thời gian dài còn làm phát sinh xung đột mã nguồn, tăng chi phí tích hợp và khiến các thay đổi khó được kiểm chứng sớm.

Sự phát triển của các phương pháp Agile, Continuous Integration, Continuous Delivery và trunk-based development đã thúc đẩy các nhóm phát triển tích hợp những thay đổi nhỏ thường xuyên hơn, đồng thời duy trì mã nguồn ở trạng thái có thể triển khai. Tuy nhiên, không phải mọi phần mã đã được tích hợp đều sẵn sàng để cung cấp ngay cho người dùng. Từ đó hình thành nhu cầu đưa mã nguồn của tính năng mới vào cùng phiên bản với hệ thống hiện tại nhưng vẫn có khả năng che giấu, vô hiệu hóa hoặc chỉ kích hoạt tính năng trong những điều kiện nhất định.

Ở dạng sơ khai, nhu cầu này có thể được giải quyết bằng các biến điều kiện, tham số dòng lệnh, biến môi trường hoặc tệp cấu hình. Những cơ chế bật, tắt hành vi phần mềm như vậy đã tồn tại từ trước khi thuật ngữ Feature Flag được sử dụng phổ biến. Khi hệ thống phát triển lớn

hơn, các cấu hình đơn giản dần được tổ chức thành một cơ chế có cấu trúc, cho phép lựa chọn giữa những luồng xử lý khác nhau tại thời điểm chạy. Cơ chế đó được gọi bằng nhiều tên như Feature Toggle, Feature Flag, Feature Switch hoặc Feature Gate. Pete Hodgson mô tả Feature Toggle là tập hợp các kỹ thuật giúp đội ngũ cung cấp chức năng mới nhanh chóng nhưng vẫn kiểm soát được mức độ an toàn; kỹ thuật này cho phép thay đổi hành vi hệ thống mà không cần sửa đổi và triển khai lại mã nguồn. [1]

Sự phổ biến của Feature Flag gắn liền với một thay đổi quan trọng trong tư duy phát triển phần mềm: deploy không đồng nghĩa với release. Deploy là quá trình đưa một phiên bản mã nguồn vào môi trường vận hành, trong khi release là quyết định cho phép người dùng thực sự tiếp cận một tính năng. Khi hai hoạt động này được tách biệt, đội phát triển có thể triển khai mã nguồn sớm nhưng giữ tính năng ở trạng thái tắt, kiểm thử tính năng trên production với một nhóm nội bộ, phát hành cho một tỷ lệ nhỏ người dùng và chỉ mở rộng phạm vi khi đã có đủ độ tin cậy.

Từ một công tắc bật hoặc tắt đơn giản, Feature Flag dần được sử dụng cho nhiều mục đích khác nhau. Release Flag hỗ trợ tích hợp mã chưa sẵn sàng mà không làm ảnh hưởng đến người dùng; Experiment Flag phục vụ A/B testing và đánh giá tác động của các phương án sản phẩm; Permission Flag giới hạn tính năng theo nhóm người dùng hoặc quyền truy cập; Ops Flag và Kill Switch giúp đội vận hành nhanh chóng vô hiệu hóa một chức năng khi phát hiện lỗi, suy giảm hiệu năng hoặc áp lực tải bất thường. Việc phân loại này cho thấy Feature Flag không chỉ là một câu lệnh điều kiện trong mã nguồn mà còn là một công cụ hỗ trợ quản lý phát hành, thử nghiệm sản phẩm và vận hành hệ thống. [1]

Tuy nhiên, khi số lượng flag, môi trường và điều kiện đánh giá tăng lên, việc quản lý bằng biến môi trường hoặc tệp cấu hình riêng lẻ trở nên khó kiểm soát. Trạng thái flag có thể không nhất quán giữa các máy chủ; người quản trị khó xác định ai đã thay đổi cấu hình; các ứng dụng phải tự cài đặt logic đánh giá; và những flag đã hết mục đích có thể tiếp tục tồn tại, tạo ra technical debt. Nghiên cứu thực nghiệm về việc sử dụng Feature Toggle trong doanh nghiệp cũng ghi nhận vai trò của các công cụ quản lý chuyên dụng, giá trị mặc định, metadata và nhật ký thay đổi đối với quá trình vận hành flag an toàn.

Những hạn chế trên tạo ra nhu cầu xây dựng một nền tảng Feature Flag tập trung. Thay vì để từng ứng dụng tự lưu trữ cấu hình và tự quyết định cách đánh giá flag, nền tảng cung cấp một nguồn cấu hình thống nhất, giao diện quản trị, cơ chế đánh giá có quy tắc, API tích hợp

và lịch sử thay đổi. Mô hình này giúp quá trình phát hành trở nên có kiểm soát, có khả năng truy vết và dễ mở rộng hơn khi số lượng ứng dụng hoặc đội phát triển tăng lên.

Xuất phát từ bối cảnh đó, đề tài tập trung nghiên cứu và xây dựng một nền tảng quản lý Feature Flag dành cho ứng dụng web. Hệ thống bao gồm dashboard quản trị để tạo và cấu hình flag; rule engine để đánh giá theo người dùng, vai trò và tỷ lệ rollout; evaluation API để các ứng dụng xác định trạng thái tính năng; audit log để ghi nhận thay đổi; cùng demo app nhằm minh họa quá trình tích hợp và sử dụng Feature Flag trong thực tế.

Việc thực hiện đề tài không chỉ nhằm xây dựng một chức năng bật hoặc tắt tính năng. Mục tiêu quan trọng hơn là nghiên cứu cách thiết kế một hệ thống có thể đưa ra quyết định nhất quán, xác định rõ thứ tự ưu tiên giữa các quy tắc, xử lý an toàn khi thiếu dữ liệu hoặc xảy ra lỗi, đồng thời cân bằng giữa tính linh hoạt, hiệu năng, khả năng kiểm soát và mức độ phức tạp của kiến trúc. Thông qua quá trình phân tích, thiết kế, triển khai và kiểm thử, đề tài hướng đến việc làm rõ giá trị của Feature Flag trong vòng đời phát triển phần mềm hiện đại, đồng thời thể hiện khả năng giải quyết vấn đề, tư duy thiết kế và khả năng giải thích các quyết định công nghệ được sử dụng trong hệ thống.

Tóm tắt nội dung và đóng góp

Báo cáo trình bày quá trình nghiên cứu Feature Flag và xây dựng một nền tảng quản lý phát hành tính năng an toàn cho ứng dụng web. Xuất phát từ vấn đề rủi ro khi triển khai tính năng mới trực tiếp cho toàn bộ người dùng, báo cáo làm rõ vai trò của Feature Flag trong việc tách biệt quá trình *deploy* mã nguồn khỏi quyết định *release* tính năng. Nội dung nghiên cứu bao gồm khái niệm Feature Flag, các loại flag phổ biến, các chiến lược phát hành như bật hoặc tắt toàn cục, targeting theo người dùng, targeting theo vai trò, rollout theo phần trăm người dùng, cùng những vấn đề vận hành như consistency, caching, auditability, bảo mật và quản lý vòng đời flag.

Trên cơ sở nghiên cứu đó, đề tài thiết kế và triển khai một hệ thống gồm Backend API, Rule Evaluation Engine, Admin Dashboard, Demo Application, JavaScript SDK và cơ sở dữ liệu PostgreSQL. Backend API cung cấp các chức năng quản lý project, feature flag, rule, flag group, sample user context, audit log, thống kê evaluation và endpoint đánh giá flag cho ứng dụng tích hợp; environment được dùng trong mô hình dữ liệu, seed data và Evaluation API thay vì có một nhóm CRUD riêng trên dashboard. Rule Evaluation Engine được thiết kế với thứ tự ưu tiên rõ ràng, bao gồm trạng thái archived, cấu hình disabled, group kill switch, flag kill switch, global on, các rule targeting và default off. Kết quả evaluation luôn trả về trạng thái bật hoặc tắt kèm reason code, giúp quyết định của hệ thống có thể giải thích và kiểm thử được.

Admin Dashboard cho phép người quản trị tạo project, cấu hình Feature Flag, sắp xếp rule, theo dõi lịch sử thay đổi và quan sát thống kê evaluation. Demo Application mô phỏng một ứng dụng khách sử dụng Evaluation API thông qua JavaScript SDK để bật hoặc ẩn tính năng tại thời điểm chạy. Các kịch bản demo thể hiện khả năng phát hành từng phần, targeting theo user hoặc role, rollout ổn định bằng stable hashing, rollback nhanh bằng kill switch và thay đổi cấu hình mà không cần triển khai lại ứng dụng.

Các đóng góp chính của đề tài bao gồm:

- Hệ thống hóa kiến thức về Feature Flag, làm rõ sự khác biệt giữa deploy và release, đồng thời phân tích lợi ích, rủi ro và phạm vi ứng dụng của Feature Flag trong quy trình phát triển phần mềm hiện đại.
- Khảo sát và so sánh các nền tảng hiện có như LaunchDarkly và Unleash để rút ra các thành phần cốt lõi cần có trong một nền tảng Feature Flag, từ đó xác định phạm vi phù hợp cho mini project.
- Thiết kế kiến trúc tách biệt giữa control plane và data plane, giúp phân định rõ phần quản trị cấu hình với phần đánh giá flag phục vụ ứng dụng tích hợp.
- Xây dựng Rule Evaluation Engine có tính xác định, hỗ trợ user targeting, role targeting, percentage rollout, kill switch và fail-closed fallback khi thiếu dữ liệu hoặc xảy ra lỗi.
- Thiết kế mô hình dữ liệu cho project, environment, Feature Flag, cấu hình theo môi trường, rule, flag group, user context mẫu, audit log và thống kê evaluation.
- Triển khai audit log dạng append-only với snapshot trước và sau thay đổi, hỗ trợ truy vết actor, action, target và lịch sử cấu hình.
- Hoàn thiện dashboard quản trị và demo application nhằm chứng minh giá trị thực tế của Feature Flag trong việc kiểm soát phạm vi phát hành, giảm rủi ro vận hành và hỗ trợ rollback nhanh.
- Bổ sung các chức năng nâng cao như cache evaluation snapshot, JavaScript SDK, phân quyền quản trị, thống kê lượt evaluation, group kill switch và Docker Compose để tăng tính hoàn chỉnh của sản phẩm.

Kết quả thực hiện cho thấy hệ thống đáp ứng được các yêu cầu cốt lõi của mini project: có báo cáo nghiên cứu, backend API, frontend dashboard, demo application, dữ liệu lưu trữ bền vững, validation, xử lý lỗi, dữ liệu mẫu, tài liệu thiết kế và các kịch bản trình diễn. Mặc dù chưa hướng tới quy mô của một nền tảng thương mại hoàn chỉnh, sản phẩm đã thể hiện đầy đủ các nguyên lý quan trọng của Feature Flag và cung cấp nền tảng có thể mở rộng trong tương lai theo các hướng như multi-variant testing, targeting theo thuộc tính, realtime configuration, SDK cho nhiều ngôn ngữ, monitoring và triển khai phân tán.

Mục lục

1 Giới thiệu **1**

1.1 Đặt vấn đề 1

1.2 Mục tiêu của báo cáo 2

1.3 Câu hỏi nghiên cứu 2

1.4 Phạm vi triển khai 3

2 Nội dung **5**

2.1 Tổng quan về Feature Flag 5

2.1.1 Khái niệm Feature Flag 5

2.1.2 Deploy và Release 6

2.1.3 Vai trò trong vòng đời phần mềm 6

2.1.4 Phân loại flag và chiến lược phát hành 7

2.1.5 Lợi ích và rủi ro 7

2.2 Khảo sát các nền tảng Feature Flag hiện có 8

2.2.1 Nền tảng LaunchDarkly 8

2.2.2 Nền tảng Unleash 8

2.2.3 So sánh và bài học rút ra 8

2.2.4 Định hướng của hệ thống đề xuất 9

2.3 Phân tích bài toán và yêu cầu hệ thống 10

2.3.1 Các đối tượng sử dụng hệ thống 10

2.3.2 Các ca sử dụng chính 10

2.3.3 Yêu cầu chức năng mức MVP 11

2.3.4 Yêu cầu phi chức năng 12

2.3.5 Các yêu cầu khuyến khích 13

2.4 Kiến trúc tổng thể của hệ thống 14
2.4.1 Các thành phần chính 14
2.4.2 Control plane và data plane 15
2.4.3 Luồng quản trị Feature Flag 17
2.4.4 Luồng đánh giá Feature Flag 17
2.4.5 Luồng tích hợp ứng dụng khách 18
2.4.6 Lựa chọn công nghệ 19
2.5 Thiết kế dữ liệu 20
2.5.1 Mô hình dữ liệu tổng thể 20
2.5.2 Project và environment 21
2.5.3 Feature Flag và cấu hình theo environment 22
2.5.4 Rule và tham số rule 22
2.5.5 User context mẫu 23
2.5.6 Flag group và group kill switch 24
2.5.7 Audit log và lịch sử cấu hình 24
2.5.8 Ràng buộc toàn vẹn dữ liệu 25
2.6 Thiết kế Rule Evaluation Engine 26
2.6.1 Evaluation request và context 26
2.6.2 Evaluation snapshot 27
2.6.3 Thứ tự ưu tiên khi đánh giá 28
2.6.4 Đánh giá trạng thái toàn cục 29
2.6.5 Targeting theo người dùng 30
2.6.6 Targeting theo vai trò 30
2.6.7 Rollout theo phần trăm người dùng 31
2.6.8 Thực thi rule theo độ ưu tiên 32
2.6.9 Variant, EvaluationResult và reason code 33
2.6.10 Default off và error fallback 34
2.6.11 Khả năng mở rộng Rule Evaluation Engine 35
2.7 Thiết kế API 35
2.7.1 Nguyên tắc thiết kế API 35
2.7.2 API quản lý project và environment 37
2.7.3 API quản lý Feature Flag 37

2.7.4 API quản lý rule . . . 38
2.7.5 API quản lý flag group và group kill switch . . . 39
2.7.6 Evaluation API . . . 39
2.7.7 API audit log và lịch sử cấu hình . . . 40
2.7.8 API thống kê evaluation . . . 41
2.7.9 Validation và mô hình lỗi . . . 41
2.7.10 Tài liệu hóa API . . . 42
2.8 Hiệu năng, nhất quán và bảo mật . . . 42
2.8.1 Caching evaluation snapshot . . . 42
2.8.2 Thiết kế cache key và thời gian tồn tại . . . 43
2.8.3 Cache invalidation . . . 43
2.8.4 Tính nhất quán của cấu hình . . . 44
2.8.5 Trust boundary của evaluation context . . . 45
2.8.6 Bảo vệ Management API . . . 46
2.8.7 Phân quyền quản trị . . . 47
2.8.8 Auditability và truy vết thay đổi . . . 47
2.9 Phương pháp kiểm thử và đánh giá . . . 47
2.9.1 Phạm vi kiểm thử . . . 48
2.9.2 Tiêu chí đánh giá Rule Evaluation Engine . . . 48
2.9.3 Tiêu chí đánh giá API và dữ liệu . . . 48
2.9.4 Tiêu chí đánh giá Dashboard và Demo Application . . . 49
2.9.5 Tiêu chí triển khai và trình diễn . . . 49

3 Kết quả thực hiện và đánh giá 50
3.1 Môi trường và cấu trúc triển khai . . . 50
3.1.1 Cấu trúc mã nguồn . . . 50
3.1.2 Môi trường phát triển . . . 52
3.1.3 Database migration và seed data . . . 53
3.1.4 Triển khai bằng Docker Compose . . . 55
3.1.5 Quy trình khởi chạy hệ thống . . . 57
3.2 Kết quả xây dựng Backend API . . . 58
3.2.1 Quản lý project và environment nội bộ . . . 60

3.2.2 Quản lý Feature Flag 60
3.2.3 Quản lý rule và flag group 61
3.2.4 Evaluation API 62
3.2.5 Audit log và lịch sử cấu hình 62
3.2.6 Validation và xử lý lỗi 63
3.2.7 Phân quyền người dùng quản trị 63
3.2.8 Cache và evaluation metrics 64
3.3 Kết quả xây dựng Frontend Dashboard 64
3.3.1 Màn hình quản lý project 65
3.3.2 Màn hình quản lý Feature Flag 66
3.3.3 Màn hình tạo và chỉnh sửa flag 66
3.3.4 Màn hình cấu hình rule 67
3.3.5 Màn hình quản lý flag group 68
3.3.6 Màn hình audit log 68
3.3.7 Màn hình thống kê evaluation 69
3.3.8 Hiển thị quyền và trạng thái 69
3.4 Kết quả xây dựng Demo Application và SDK 70
3.4.1 Cấu trúc Demo Application 70
3.4.2 JavaScript SDK 71
3.4.3 Tích hợp SDK với Demo Application 71
3.4.4 User context trong Demo Application 72
3.4.5 Hành vi fallback 72
3.5 Các kịch bản minh họa 73
3.5.1 Bật và tắt tính năng toàn cục 74
3.5.2 Targeting theo user và role 75
3.5.3 Rollout theo phần trăm 75
3.5.4 Kill switch ở cấp flag và group 75
3.5.5 Audit log, history và statistics 75
3.5.6 Fallback của SDK 76
3.6 Kết quả kiểm thử 76
3.6.1 Tổng quan kết quả 76
3.6.2 Rule Evaluation Engine 76

3.6.3 Backend API và persistence . . . 77
3.6.4 Cache, statistics và RBAC . . . 77
3.6.5 Dashboard, Demo Application và Docker Compose . . . 77
3.7 Đánh giá kết quả thực hiện . . . 78
3.7.1 Mức độ đáp ứng yêu cầu . . . 78
3.7.2 Đánh giá thiết kế kỹ thuật . . . 78
3.7.3 Đánh giá giá trị thực tế . . . 78
3.7.4 Hạn chế còn lại . . . 79

4 Kết luận 80
4.1 Tổng kết kết quả đạt được . . . 80
4.2 Các đóng góp chính . . . 81
4.2.1 Đóng góp về nghiên cứu . . . 81
4.2.2 Đóng góp về thiết kế . . . 81
4.2.3 Đóng góp về triển khai . . . 81
4.2.4 Đóng góp về tư duy hệ thống . . . 81
4.3 Bài học kinh nghiệm . . . 82
4.3.1 Bài học về phân tích bài toán . . . 82
4.3.2 Bài học về thiết kế hệ thống . . . 82
4.3.3 Bài học về triển khai và kiểm thử . . . 83
4.4 Hạn chế . . . 83
4.4.1 Hạn chế về kiến trúc . . . 83
4.4.2 Hạn chế về chức năng . . . 84
4.4.3 Hạn chế về bảo mật và vận hành . . . 84
4.5 Hướng phát triển . . . 85
4.6 Mở rộng Rule Evaluation Engine . . . 85
4.6.1 Targeting theo thuộc tính . . . 85
4.6.2 Targeting theo organization . . . 85
4.6.3 Targeting theo thời gian và vị trí . . . 85
4.6.4 Hỗ trợ nhiều variant và A/B testing . . . 85
4.7 Mở rộng khả năng tích hợp . . . 86
4.7.1 SDK cho nhiều ngôn ngữ . . . 86

4.7.2 Webhook khi cấu hình thay đổi 86
4.7.3 Đồng bộ cấu hình theo thời gian thực 86
4.7.4 Tương thích OpenFeature 86
4.8 Mở rộng quản trị vòng đời flag 86
4.8.1 Approval workflow 86
4.8.2 Lập lịch bật và tắt flag 86
4.8.3 Phát hiện stale flag 86
4.8.4 Nhắc nhở và tự động dọn dẹp flag 86
4.9 Mở rộng khả năng vận hành 87
4.9.1 High availability 87
4.9.2 Distributed cache 87
4.9.3 Monitoring và alerting 87
4.9.4 Triển khai trên Kubernetes 87

Tài liệu tham khảo **88**

Danh sách hình vẽ

2.1 Kiến trúc tổng thể của hệ thống Feature Flag Platform 14
2.2 Phân tách control plane và data plane trong hệ thống 16
2.3 Luồng đánh giá Feature Flag từ ứng dụng khách đến Rule Evaluation Engine . . 18
2.4 Sơ đồ thực thể liên kết rút gọn của hệ thống Feature Flag 21
2.5 Flowchart thứ tự ưu tiên của Rule Evaluation Engine 28
2.6 Luồng cache invalidation và bảo đảm tính nhất quán cấu hình 44

3.1 Các service chính trong quy trình triển khai bằng Docker Compose 55
3.2 Minh chứng giao diện tài liệu Backend API/OpenAPI đã được sinh từ source code 59
3.3 Minh chứng giao diện Admin Dashboard đã được xây dựng 65

Danh sách bảng

2.1 So sánh deploy và release 6
2.2 Đặc điểm vòng đời của một số loại Feature Flag 7
2.3 Tổng hợp so sánh LaunchDarkly và Unleash 9
2.4 Các thực thể và quan hệ chính trong mô hình dữ liệu 20
2.5 Các ràng buộc duy nhất tiêu biểu 25
2.6 Tương tác giữa trạng thái và serving mode 30
2.7 Reason code cho trạng thái hệ thống 34
2.8 Quan hệ giữa mã lý do và matched rule 34

3.1 Tổng hợp các nhóm endpoint đã triển khai trong Backend API 59
3.2 Bảng kịch bản minh họa end-to-end của hệ thống 74
3.3 Tổng hợp kết quả kiểm thử tự động sau khi chạy `npm run test - -runInBand` . 76

Danh sách đoạn mã

2.1 Điều khiển hành vi ứng dụng bằng Feature Flag 5
2.2 Cấu trúc evaluation request 26
2.3 Thứ tự ưu tiên của Rule Evaluation Engine 28
2.4 Tạo giá trị băm ổn định bằng SHA-256 31
2.5 Chuyển đổi SHA-256 digest thành bucket phần trăm 32
2.6 Cấu trúc EvaluationResult 33
2.7 Tập reason code của Evaluation API 33
2.8 Kết quả fallback mặc định 34
2.9 Xử lý lỗi nội bộ của Evaluation Engine 35

Chương 1

Giới thiệu

1.1 Đặt vấn đề

Trong quá trình phát triển phần mềm, việc đưa một tính năng mới vào hệ thống luôn đi kèm với rủi ro. Mặc dù tính năng đã được kiểm thử trước khi triển khai, các vấn đề về logic nghiệp vụ, hiệu năng, khả năng tương thích hoặc hành vi của người dùng vẫn có thể chỉ xuất hiện trên môi trường thực tế. Nếu tính năng được phát hành đồng thời cho toàn bộ người dùng, phạm vi ảnh hưởng của sự cố có thể lớn, trong khi việc khắc phục thường đòi hỏi rollback phiên bản hoặc thực hiện một lần triển khai mới.

Bên cạnh đó, nhu cầu phát hành phần mềm hiện nay không còn giới hạn ở hai trạng thái bật hoặc tắt cho toàn hệ thống. Đội phát triển có thể cần cung cấp tính năng cho một số người dùng cụ thể, một nhóm có vai trò nhất định hoặc một tỷ lệ người dùng được lựa chọn ổn định. Do đó, bài toán đặt ra là xây dựng một cơ chế cho phép kiểm soát việc phát hành tính năng độc lập với quá trình triển khai mã nguồn, đồng thời bảo đảm quyết định bật hoặc tắt tính năng được thực hiện nhất quán, có thể truy vết và dễ dàng quản lý.

Feature Flag là một giải pháp phù hợp cho bài toán này. Tuy nhiên, khi số lượng flag và quy tắc tăng lên, việc sử dụng các biến cấu hình rời rạc hoặc cài đặt logic trực tiếp trong từng ứng dụng có thể dẫn đến cấu hình không nhất quán, khó kiểm soát lịch sử thay đổi và khó mở rộng. Vì vậy, đề tài lựa chọn nghiên cứu và xây dựng một nền tảng quản lý Feature Flag tập trung cho ứng dụng web.

1.2 Mục tiêu của báo cáo

Báo cáo hướng đến hai nhóm mục tiêu chính: mục tiêu nghiên cứu và mục tiêu triển khai.

Về nghiên cứu, báo cáo làm rõ bản chất của Feature Flag, sự khác biệt giữa deploy và release, các loại flag phổ biến và vai trò của Feature Flag trong quản lý phát hành phần mềm. Báo cáo đồng thời phân tích các chiến lược đánh giá như bật toàn cục, nhắm mục tiêu theo người dùng, theo vai trò và rollout theo tỷ lệ phần trăm.

Về triển khai, đề tài xây dựng một hệ thống có khả năng quản lý project, feature flag và các rule tương ứng; tiếp nhận evaluation context từ ứng dụng tích hợp; đánh giá trạng thái flag theo một thứ tự ưu tiên xác định; trả về kết quả có lý do rõ ràng; và ghi nhận các thay đổi cấu hình thông qua audit log. Hệ thống cũng cung cấp dashboard quản trị và demo app để chứng minh khả năng ứng dụng trong thực tế. Các thành phần này phù hợp với yêu cầu về backend API, frontend dashboard, evaluation engine, audit log và ứng dụng minh họa của mini project.

Ngoài việc bảo đảm hệ thống có thể hoạt động, báo cáo còn tập trung giải thích các quyết định thiết kế và công nghệ, bao gồm cách tổ chức kiến trúc, mô hình dữ liệu, thứ tự đánh giá rule, cơ chế rollout ổn định, xử lý lỗi và khả năng mở rộng. Đây cũng là cơ sở để thể hiện khả năng giải quyết vấn đề, tư duy thiết kế và mức độ hiểu những nội dung đã triển khai.

1.3 Câu hỏi nghiên cứu

Báo cáo tập trung giải quyết các câu hỏi chính sau:

1. Feature Flag giải quyết những hạn chế nào của quy trình triển khai và phát hành phần mềm truyền thống?
2. Làm thế nào để tách quá trình deploy mã nguồn khỏi quyết định release tính năng cho người dùng?
3. Một nền tảng Feature Flag cần những thành phần nào để hỗ trợ quản lý cấu hình, đánh giá rule, tích hợp với ứng dụng và truy vết thay đổi?
4. Làm thế nào để xác định thứ tự ưu tiên giữa các trạng thái và rule nhằm bảo đảm kết quả đánh giá rõ ràng, nhất quán và có thể dự đoán?
5. Làm thế nào để rollout tính năng theo tỷ lệ phần trăm mà vẫn giữ kết quả ổn định đối

với cùng một người dùng?

6. Hệ thống nên xử lý như thế nào khi feature flag không tồn tại, dữ liệu context bị thiếu hoặc xảy ra lỗi trong quá trình đánh giá?

7. Kiến trúc và công nghệ nào phù hợp với quy mô của mini project, đồng thời vẫn tạo điều kiện để hệ thống được mở rộng trong tương lai?

1.4 Phạm vi triển khai

Trong phạm vi báo cáo, đề tài triển khai một nền tảng Feature Flag phục vụ các ứng dụng web, gồm bốn thành phần chính:

- Backend API chịu trách nhiệm quản lý dữ liệu, xử lý nghiệp vụ và cung cấp evaluation endpoint.
- Frontend dashboard hỗ trợ quản trị project, flag, rule, group, audit log và thống kê evaluation.
- Evaluation engine thực hiện việc đánh giá flag dựa trên trạng thái cấu hình và context của request.
- Demo app tích hợp evaluation API để minh họa việc hiển thị hoặc ẩn tính năng theo kết quả đánh giá.

Các chiến lược được hỗ trợ bao gồm bật hoặc tắt toàn cục, nhắm mục tiêu theo user ID, theo role và rollout theo phần trăm người dùng. Hệ thống lưu trữ project, môi trường, feature flag, cấu hình, rule, dữ liệu người dùng mẫu và audit log trong cơ sở dữ liệu. Ngoài ra, hệ thống thực hiện validation dữ liệu đầu vào, xử lý lỗi cơ bản, cung cấp dữ liệu mẫu và tài liệu hướng dẫn cài đặt.

Đề tài tập trung vào việc chứng minh đầy đủ luồng hoạt động của Feature Flag, từ khâu tạo và cấu hình flag đến khi ứng dụng gửi context và nhận kết quả đánh giá. Các vấn đề ở quy mô hệ thống phân tán lớn như đồng bộ cấu hình đa vùng, streaming update theo thời gian thực, độ sẵn sàng đa trung tâm dữ liệu, phân tích thử nghiệm thống kê chuyên sâu và quản lý hàng triệu request mỗi giây không thuộc phạm vi triển khai chính.

Các nội dung nâng cao như caching, SDK JavaScript, phân quyền, lịch sử cấu hình dựa trên audit log, thống kê lượt đánh giá, kill switch theo nhóm và Docker Compose được xem là phần

mở rộng nhằm đánh giá khả năng phát triển tiếp của hệ thống, thay vì điều kiện bắt buộc để hoàn thành luồng chức năng cốt lõi.

Chương 2

Nội dung

2.1 Tổng quan về Feature Flag

2.1.1 Khái niệm Feature Flag

Feature Flag, còn gọi là Feature Toggle hoặc Feature Switch, là kỹ thuật cho phép phần mềm quyết định tại thời điểm chạy liệu một tính năng hoặc một nhánh xử lý có được kích hoạt hay không. Quyết định này dựa trên cấu hình bên ngoài mã nguồn và evaluation context của đối tượng đang sử dụng hệ thống [1, 2].

Một flag thường có các thành phần cốt lõi: khóa định danh, trạng thái hoặc cấu hình mặc định, rule targeting, chiến lược rollout, evaluation context và giá trị trả về. Trong một nền tảng quản lý tập trung, các flag còn được đặt trong phạm vi project và environment để cùng một tính năng có thể có cấu hình khác nhau giữa development, staging và production [3].

Ở runtime, ứng dụng gọi SDK hoặc Evaluation API bằng flag key và context. Kết quả trả về được dùng để chọn nhánh xử lý trong mã nguồn đã deploy:

Đoạn mã 2.1: Điều khiển hành vi ứng dụng bằng Feature Flag

```
const result = await featureFlagClient.evaluate(
    "new-checkout-page",
    userContext
);

if (result.enabled) {
```

```
    return showNewCheckout();
}

return showCurrentCheckout();
```

Điểm quan trọng là Feature Flag không tự thay đổi mã nguồn; nó chỉ thay đổi cấu hình quyết định nhánh xử lý nào được sử dụng.

2.1.2 Deploy và Release

Deploy là quá trình đưa phiên bản phần mềm lên môi trường chạy. Release là quyết định cho phép người dùng thực sự sử dụng một tính năng. Feature Flag có giá trị vì nó tách hai hoạt động này: mã có thể đã được deploy nhưng tính năng vẫn chưa được release rộng rãi [4].

Sự khác biệt giữa deploy và release được thể hiện trong Bảng 2.1.

Bảng 2.1: So sánh deploy và release

Tiêu chí	**Deploy**	**Release**
Mục tiêu	Đưa mã nguồn lên môi trường chạy	Cung cấp tính năng cho người dùng
Phạm vi tác động	Hạ tầng, dịch vụ, phiên bản ứng dụng	Người dùng hoặc nhóm người dùng
Cách thực hiện	CI/CD, script triển khai, hạ tầng	Cấu hình flag hoặc rule targeting
Đảo ngược	Có thể cần rollback phiên bản	Có thể tắt hoặc thu hẹp flag

Khi deploy và release bị gắn chặt, mỗi lần đưa mã lên production đều có thể làm thay đổi hành vi của toàn bộ hệ thống. Khi tách chúng bằng Feature Flag, đội phát triển có thể kiểm thử có kiểm soát, mở rộng dần phạm vi phát hành và tắt nhanh tính năng nếu phát sinh sự cố [1].

2.1.3 Vai trò trong vòng đời phần mềm

Trong vòng đời phần mềm, Feature Flag được dùng như một cơ chế điều phối release chứ không chỉ là câu lệnh điều kiện. Các vai trò chính gồm:

- tích hợp sớm mã chưa hoàn thiện vào nhánh chính mà chưa hiển thị cho người dùng;
- kiểm thử tính năng trên môi trường thật với nhóm nội bộ hoặc khách hàng beta;
- mở rộng phát hành theo từng giai đoạn thay vì bật cho toàn bộ người dùng ngay lập tức;

- rollback runtime bằng cách tắt flag khi tính năng gây lỗi hoặc ảnh hưởng hiệu năng;
- ghi nhận lịch sử cấu hình để truy vết nguyên nhân khi hành vi ứng dụng thay đổi.

Các vai trò trên là cơ sở để mini project tập trung vào quản lý cấu hình, evaluation có precedence rõ ràng, audit log và demo runtime thay vì chỉ xây dựng chức năng bật/tắt đơn giản.

2.1.4 Phân loại flag và chiến lược phát hành

Các loại flag phổ biến khác nhau ở mục đích và thời gian tồn tại. Release flag thường ngắn hạn và phục vụ phát hành tính năng mới. Experiment flag dùng cho thử nghiệm sản phẩm. Ops hoặc kill-switch flag phục vụ vận hành và xử lý sự cố. Permission flag giới hạn tính năng theo nhóm người dùng hoặc gói dịch vụ [1, 5].

Bảng 2.2: Đặc điểm vòng đời của một số loại Feature Flag

Loại flag	**Mục đích chính**	**Thời gian tồn tại**	**Rủi ro chính**
Release flag	Phát hành tính năng mới	Ngắn hạn	Quên dọn dẹp
Experiment flag	A/B testing hoặc đo lường	Theo thời gian thử nghiệm	Sai lệch dữ liệu
Ops/Kill switch	Tắt nhanh khi có sự cố	Dài hơn, tùy vận hành	Bị lạm dụng thay cấu hình
Permission flag	Giới hạn theo nhóm người dùng	Theo chính sách sản phẩm	Nhầm với authorization

Về chiến lược release, hệ thống mini project tập trung vào bốn trường hợp đủ để minh họa giá trị cốt lõi: bật/tắt toàn cục, user allowlist, role targeting và percentage rollout. Percentage rollout cần stable hashing để cùng một targeting key nhận kết quả ổn định khi cấu hình không đổi [6, 7].

2.1.5 Lợi ích và rủi ro

Lợi ích chính của Feature Flag là giảm rủi ro phát hành: đội phát triển có thể deploy thường xuyên, release có kiểm soát, rollback nhanh và quan sát tác động qua audit/statistics. Tuy nhiên, cùng một cơ chế cũng tạo thêm trách nhiệm vận hành.

Các rủi ro cần quản lý gồm feature flag debt, cấu hình sai, rule chồng chéo khó giải thích, context chứa dữ liệu nhạy cảm và việc dùng flag thay cho cơ chế authorization thật. Vì vậy, nền tảng Feature Flag cần validation, precedence rõ ràng, safe default, audit log, phân quyền quản trị và quy trình dọn dẹp flag sau khi hết mục đích [8, 9, 10].

2.2 Khảo sát các nền tảng Feature Flag hiện có

Phần này khảo sát hai nền tảng tiêu biểu là LaunchDarkly và Unleash nhằm rút ra các nguyên tắc thiết kế cần kế thừa cho mini project. Nội dung chỉ tập trung vào điểm khác biệt và bài học thiết kế, tránh lặp lại các khái niệm Feature Flag đã trình bày ở phần tổng quan.

2.2.1 Nền tảng LaunchDarkly

LaunchDarkly là nền tảng Feature Flag thương mại dạng dịch vụ được quản lý. Nền tảng tổ chức tài nguyên theo project, environment và flag; cung cấp SDK đa nền tảng; hỗ trợ targeting rule có thứ tự, percentage rollout, nhiều variation, audit log và quản trị quyền truy cập [3, 6].

Điểm đáng chú ý đối với mini project là mô hình ordered rule. Người quản trị có thể đặt rule cụ thể trước rule tổng quát, ví dụ user allowlist trước percentage rollout. Cách biểu diễn này giúp kết quả evaluation dễ giải thích vì engine dừng tại rule đầu tiên khớp.

Hạn chế chính khi tham chiếu LaunchDarkly cho mini project là phạm vi tính năng rất rộng. Các khả năng như segment phức tạp, nhiều variation, workflow phê duyệt và phân tích thử nghiệm chuyên sâu được xem là hướng mở rộng, không phải yêu cầu triển khai ban đầu.

2.2.2 Nền tảng Unleash

Unleash là nền tảng Feature Flag mã nguồn mở, có thể self-hosted hoặc dùng dịch vụ được quản lý. Nền tảng sử dụng activation strategy để quyết định flag có được bật hay không trong từng environment; SDK thường tải cấu hình về và đánh giá tại ứng dụng [2, 5].

Điểm đáng chú ý đối với mini project là khả năng tự triển khai và tư duy tách control plane khỏi runtime evaluation. Unleash cũng nhấn mạnh lifecycle của flag, metric evaluation và việc tránh để hệ thống Feature Flag trở thành phụ thuộc đồng bộ bắt buộc của mọi request nghiệp vụ.

Hạn chế của mô hình self-hosted là tổ chức phải tự chịu trách nhiệm về database, backup, monitoring, nâng cấp và mở rộng. Vì mini project cần dễ chạy và dễ trình diễn, hệ thống chỉ mô phỏng phần lõi bằng một backend duy nhất, PostgreSQL và cache đơn giản.

2.2.3 So sánh và bài học rút ra

Bảng 2.3 tóm tắt các điểm so sánh có ảnh hưởng trực tiếp đến thiết kế mini project.

Bảng 2.3: Tổng hợp so sánh LaunchDarkly và Unleash

Tiêu chí	**LaunchDarkly**	**Unleash**
Mô hình triển khai	Dịch vụ được quản lý	Managed hoặc self-hosted
Đơn vị logic	Ordered targeting rule	Activation strategy và constraint
Evaluation	Chủ yếu qua SDK	Chủ yếu qua SDK hoặc proxy/edge
Rollout	Percentage rollout ổn định	Gradual rollout ổn định
Audit/RBAC	Tích hợp phong phú	Phụ thuộc phiên bản/mô hình triển khai
Bài học cho mini project	Precedence rõ, reason dễ giải thích	Tách plane, self-hosted workflow, lifecycle

Từ khảo sát, mini project kế thừa các nguyên tắc sau:

- tổ chức tài nguyên theo project, environment và flag;
- tách control plane quản trị khỏi data plane evaluation;
- dùng evaluation context thay vì hard-code điều kiện trong ứng dụng;
- áp dụng stable rollout bằng khóa không chứa PII;
- trả reason code để giải thích kết quả evaluation;
- ghi audit log cho mọi mutation quan trọng;
- cung cấp SDK và fallback an toàn cho ứng dụng tích hợp.

2.2.4 Định hướng của hệ thống đề xuất

Hệ thống đề xuất không cố gắng sao chép đầy đủ LaunchDarkly hoặc Unleash. Phạm vi được chọn là một lát cắt đủ nhỏ để triển khai trong mini project nhưng vẫn thể hiện được bản chất của nền tảng Feature Flag: quản lý cấu hình, đánh giá runtime, thay đổi hành vi ứng dụng, auditability và trình diễn release an toàn.

Các chức năng được kế thừa trực tiếp gồm project/environment, flag config theo environment, ordered rule, user/role targeting, percentage rollout, kill switch, audit log, statistics, SDK và Docker Compose. Các chức năng được đơn giản hóa gồm multi-variant testing, segment phức tạp, approval workflow, realtime streaming, multi-region deployment và phân quyền enterprise.

Điểm khác biệt của MiniProject là ưu tiên tính minh bạch và khả năng giải thích hơn độ rộng

tính năng. Vì vậy, Rule Evaluation Engine dùng precedence cố định, response có reason code rõ ràng, audit log append-only và Dashboard tập trung vào các kịch bản demo quan trọng nhất.

2.3 Phân tích bài toán và yêu cầu hệ thống

Mục này xác định phạm vi bài toán trước khi đi vào thiết kế. Các mô tả dưới đây dừng ở mức yêu cầu; chi tiết kiến trúc, dữ liệu, API và kiểm thử được trình bày ở các mục riêng để tránh trùng lặp.

2.3.1 Các đối tượng sử dụng hệ thống

Hệ thống phục vụ bốn nhóm tác nhân: người quản trị, nhà phát triển, người xem và ứng dụng tích hợp Feature Flag.

Quản trị viên

Quản trị viên chịu trách nhiệm tạo project, cấu hình flag, quản lý rule/group, kích hoạt kill switch và theo dõi lịch sử thay đổi.

Nhà phát triển

Nhà phát triển sử dụng nền tảng để chuẩn bị rollout, kiểm thử cấu hình và tích hợp Evaluation API hoặc SDK vào ứng dụng.

Người xem

Người xem cần quan sát trạng thái flag, audit log và thống kê nhưng không được thực hiện mutation cấu hình.

Ứng dụng tích hợp Feature Flag

Ứng dụng tích hợp gửi evaluation request kèm context và sử dụng kết quả trả về để bật hoặc ẩn hành vi tương ứng.

2.3.2 Các ca sử dụng chính

Các ca sử dụng được gom theo chuỗi thao tác từ quản trị đến runtime evaluation.

Quản lý project và environment

Người dùng cần tạo project và sử dụng environment làm phạm vi cấu hình/evaluation.

Quản lý Feature Flag

Hệ thống cần hỗ trợ tạo, cập nhật, archive/restore và xem trạng thái của Feature Flag.

Cấu hình chiến lược rollout

Người dùng cần cấu hình serving mode, user allowlist, role targeting, percentage rollout và kill switch.

Đánh giá Feature Flag

Ứng dụng khách cần nhận quyết định bật/tắt có reason code từ Evaluation API.

Theo dõi thay đổi và thống kê

Hệ thống cần ghi lại mutation cấu hình và tổng hợp số lượt evaluation để hỗ trợ truy vết và trình diễn.

2.3.3 Yêu cầu chức năng mức MVP

MVP tập trung vào một luồng end-to-end hoàn chỉnh: quản trị flag, lưu dữ liệu, evaluate từ ứng dụng demo và kiểm chứng kết quả.

Yêu cầu đối với Backend API

Backend phải cung cấp Management API, Evaluation API, validation, lỗi chuẩn hóa, persistence, audit log và seed data.

Yêu cầu đối với Frontend Dashboard

Dashboard phải cho phép quản lý project/flag/rule, quan sát trạng thái, xem audit log và thống kê ở mức đủ cho demo.

Yêu cầu đối với Demo Application

Demo Application phải cho thấy thay đổi cấu hình trên Dashboard ảnh hưởng đến hành vi ứng dụng mà không cần deploy lại.

Yêu cầu đối với dữ liệu và tài liệu

Sản phẩm cần có schema/migration, dữ liệu mẫu, tài liệu thiết kế ngắn và hướng dẫn chạy để phục vụ chấm điểm.

2.3.4 Yêu cầu phi chức năng

Các yêu cầu phi chức năng bảo đảm hệ thống không chỉ chạy được mà còn có hành vi đáng tin cậy khi cấu hình phức tạp hơn.

Tính đúng đắn và xác định của kết quả

Cùng một cấu hình và context phải cho cùng một kết quả; percentage rollout phải dùng stable hashing.

Hiệu năng evaluation

Evaluation API cần có độ trễ thấp và có thể tận dụng snapshot cache mà không làm sai quyết định.

Tính nhất quán của cấu hình

Mutation cấu hình phải làm mới dữ liệu runtime liên quan và tránh để engine dùng snapshot lỗi thời quá lâu.

Khả năng bảo trì và mở rộng

Logic evaluation cần tách khỏi controller để có thể bổ sung rule hoặc SDK mới trong tương lai.

Khả năng kiểm thử

Các nhánh precedence, fallback, validation và persistence phải có test tự động hoặc kịch bản kiểm chứng rõ ràng.

Bảo mật và khả năng truy vết

Management API cần RBAC demo, audit log append-only và không tin tưởng dữ liệu context do client tự khai báo.

2.3.5 Các yêu cầu khuyến khích

Ngoài MVP, đề tài bổ sung các năng lực giúp sản phẩm gần hơn với nền tảng Feature Flag thực tế.

Caching evaluation snapshot

Cache snapshot cấu hình để giảm truy vấn lặp của Evaluation API.

JavaScript SDK

SDK giúp ứng dụng web gọi Evaluation API bằng một giao diện ổn định và fail-closed.

Unit test cho Rule Evaluation Engine

Engine cần test cho precedence, stable rollout và các trường hợp lỗi/thiếu context.

Versioning và lịch sử cấu hình

Lịch sử cấu hình được dựng từ audit log thay vì bảng version riêng.

Phân quyền người dùng quản trị

RBAC demo phân biệt admin, developer và viewer cho các thao tác quản trị.

Thống kê số lần evaluation

Hệ thống tổng hợp số lượt evaluation theo flag, kết quả và thời gian.

Group kill switch

Group kill switch cho phép vô hiệu hóa nhanh nhiều flag liên quan.

Triển khai bằng Docker Compose

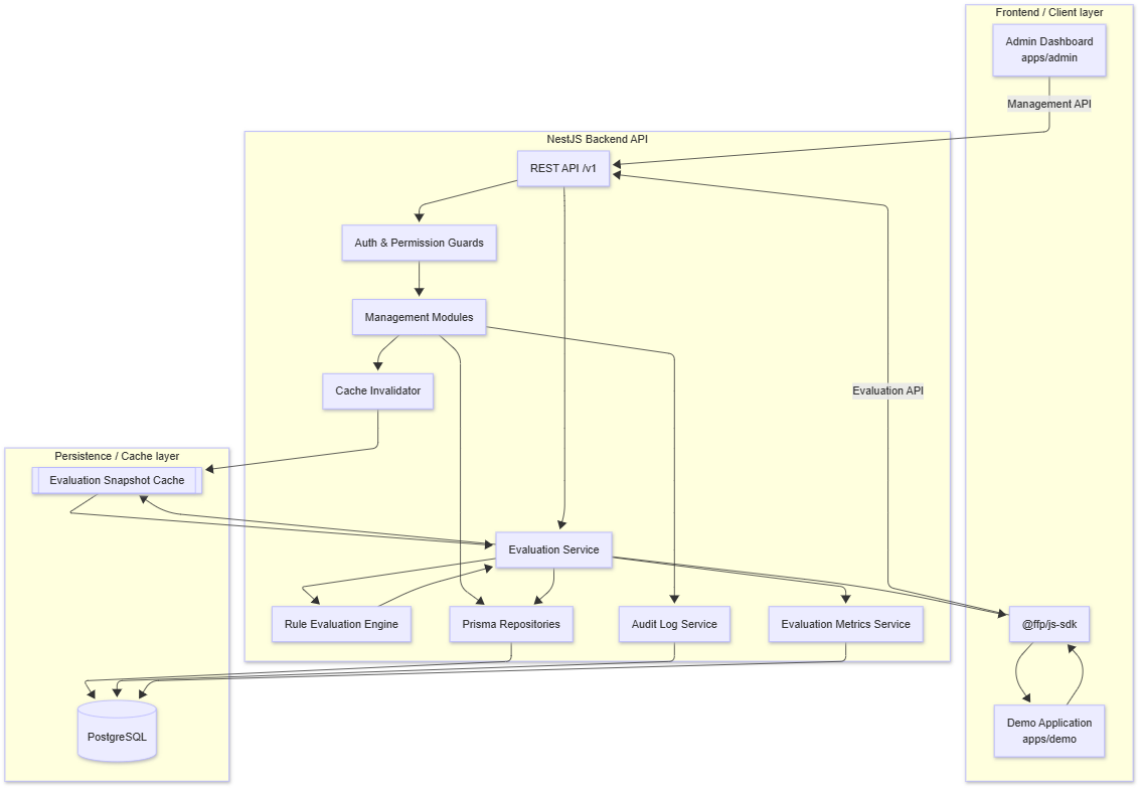
Docker Compose cung cấp quy trình khởi chạy thống nhất cho database, backend, dashboard và demo.

2.4 Kiến trúc tổng thể của hệ thống

Phần này trình bày kiến trúc ở mức hệ thống. Các chi tiết về mô hình dữ liệu, Rule Evaluation Engine, API, cache và bảo mật được tách sang các mục chuyên sâu phía sau để tránh lặp lại nội dung.

2.4.1 Các thành phần chính

Hệ thống được tổ chức thành sáu thành phần: Admin Dashboard, Backend API, Rule Evaluation Engine, PostgreSQL/Cache, JavaScript SDK và Demo Application. Dashboard phục vụ quản trị cấu hình; Backend API là ranh giới truy cập duy nhất; Rule Evaluation Engine quyết định trạng thái flag tại runtime; database lưu cấu hình và audit log; cache tăng tốc truy xuất snapshot; SDK và Demo Application minh họa cách ứng dụng khách tích hợp Feature Flag.



Hình 2.1: Kiến trúc tổng thể của hệ thống Feature Flag Platform

Admin Dashboard

Dashboard cung cấp giao diện quản trị cho project, Feature Flag, rule, flag group, audit log, history, RBAC demo và thống kê evaluation. Dashboard không tự quyết định kết quả evaluation mà gửi yêu cầu đến Backend API, nhờ đó logic nghiệp vụ và quyền truy cập vẫn được kiểm

soát tập trung ở phía máy chủ.

Backend API

Backend API gom toàn bộ management endpoint và Evaluation API trong một service NestJS. Các endpoint quản trị xử lý validation, phân quyền, persistence và audit log; Evaluation API phục vụ ứng dụng tích hợp với response nhất quán gồm trạng thái bật/tắt và reason code.

Rule Evaluation Engine

Rule Evaluation Engine là lớp domain thuần, độc lập với controller. Engine nhận evaluation request, sử dụng snapshot cấu hình đã được chuẩn hóa và trả về quyết định xác định. Thứ tự ưu tiên chi tiết được trình bày trong mục thiết kế engine, vì vậy mục này chỉ xác định vị trí của engine trong kiến trúc tổng thể.

Database và cache

PostgreSQL là nguồn dữ liệu chính cho project, environment, flag, rule, group, audit log và thống kê aggregate. Cache chỉ được xem là tầng tăng tốc cho evaluation snapshot; dữ liệu trong cache có thể bị loại bỏ mà không làm mất tính đúng đắn vì database vẫn là nguồn sự thật.

JavaScript SDK

JavaScript SDK đóng gói cách gọi Evaluation API, chuẩn hóa timeout, lỗi mạng và fallback fail-closed cho ứng dụng web. SDK giúp Demo Application không phụ thuộc trực tiếp vào chi tiết endpoint và là cơ sở để mở rộng SDK cho các ngôn ngữ khác.

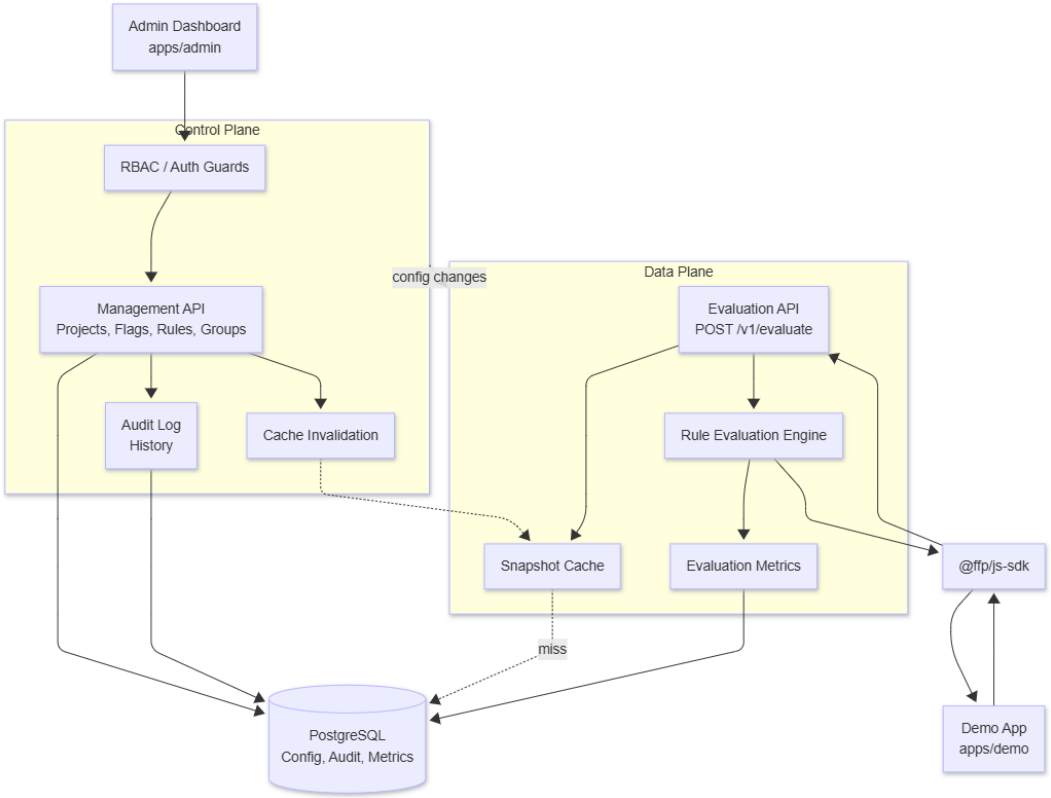
Demo Application

Demo Application mô phỏng một hệ thống thương mại điện tử nhỏ, sử dụng SDK để quyết định hiển thị tính năng theo kết quả evaluation. Vai trò của ứng dụng này là chứng minh giá trị thực tế của nền tảng thay vì chứa logic quản trị riêng.

2.4.2 Control plane và data plane

Kiến trúc được phân chia theo hai luồng trách nhiệm. *Control plane* quản lý cấu hình và lịch sử thay đổi; *data plane* xử lý evaluation ở runtime. Cách phân chia này phù hợp với đặc điểm

của Feature Flag: thao tác quản trị ít hơn nhưng yêu cầu evaluation có thể xuất hiện liên tục từ ứng dụng tích hợp [3, 5].



Hình 2.2: Phân tách control plane và data plane trong hệ thống

Vai trò của control plane

Control plane bao gồm Dashboard, management endpoints, persistence và audit logging. Mọi mutation quan trọng đều đi qua validation, kiểm tra quyền, ghi database và tạo audit entry append-only.

Vai trò của data plane

Data plane bao gồm Evaluation API, Rule Evaluation Engine và snapshot cache. Luồng này được tối ưu cho quyết định nhanh, xác định và fail-closed khi thiếu cấu hình hoặc phát sinh lỗi.

Luồng dữ liệu giữa hai thành phần

Người quản trị thay đổi cấu hình qua Dashboard; Backend API lưu thay đổi, ghi audit log và invalidation snapshot liên quan. Ứng dụng khách gửi evaluation context qua SDK hoặc gọi trực tiếp Evaluation API; engine đọc snapshot hiện tại và trả kết quả cho ứng dụng.

Lợi ích của việc tách biệt hai plane

Sự tách biệt giúp giảm coupling giữa giao diện quản trị và runtime evaluation, cho phép tối ưu hai luồng theo tải khác nhau, đồng thời làm rõ ranh giới bảo mật: dữ liệu quản trị và lịch sử thay đổi không cần phơi bày cho ứng dụng khách.

2.4.3 Luồng quản trị Feature Flag

Luồng quản trị tập trung vào việc biến ý định phát hành thành cấu hình có kiểm soát. Các bước dưới đây chỉ mô tả trình tự kiến trúc; cấu trúc bảng dữ liệu và endpoint cụ thể được trình bày ở các mục sau.

Tạo và chỉnh sửa Feature Flag

Người quản trị khai báo flag key, tên và mô tả. Backend kiểm tra ràng buộc duy nhất trong project, tạo hoặc cập nhật bản ghi liên quan và trả kết quả cho Dashboard.

Cấu hình trạng thái và serving mode

Sau khi có flag, người quản trị cấu hình lifecycle/status và serving mode theo từng environment. Thiết kế này tách trạng thái quản lý của flag khỏi cách flag được phục vụ ở runtime.

Cấu hình và sắp xếp rule

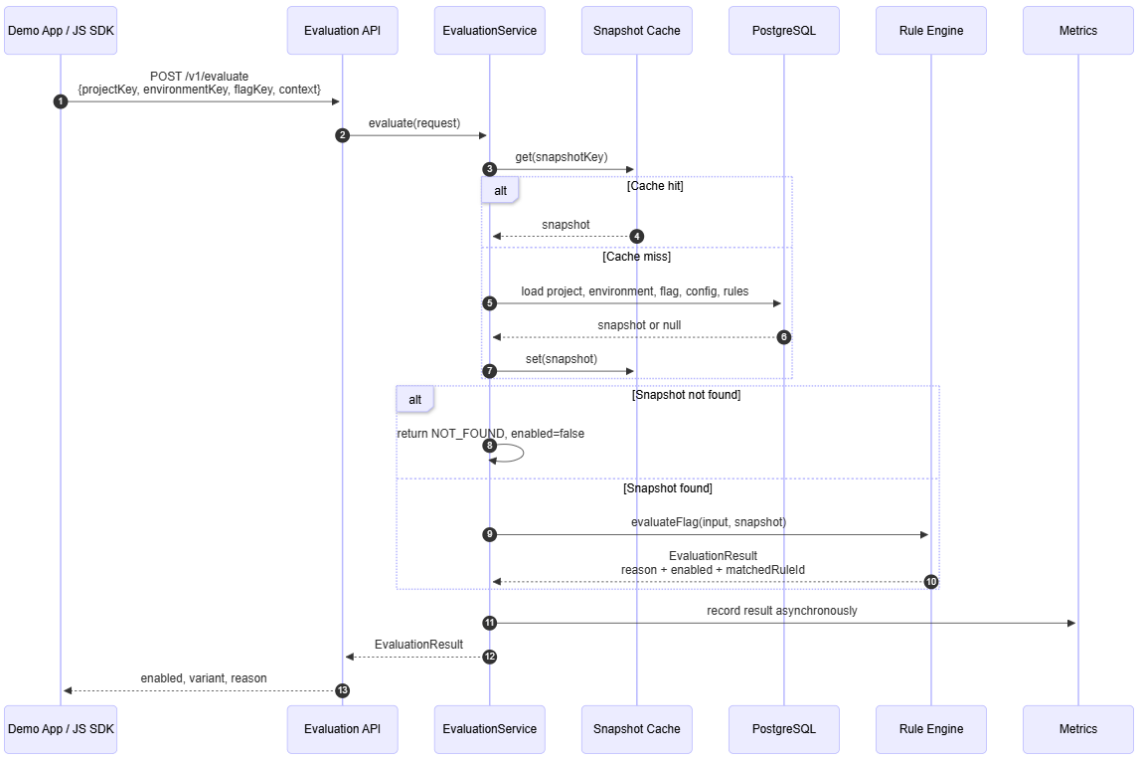
Khi flag chạy ở chế độ targeted, người quản trị cấu hình các rule hỗ trợ bởi engine. Dashboard chỉ biểu diễn và gửi cấu hình; việc diễn giải rule thuộc trách nhiệm của Rule Evaluation Engine.

Lưu audit log và invalidation cache

Sau mỗi mutation, Backend API ghi audit log và làm mới snapshot liên quan. Nhờ vậy, hệ thống vừa có khả năng truy vết vừa hạn chế việc data plane sử dụng cấu hình cũ.

2.4.4 Luồng đánh giá Feature Flag

Luồng evaluation bắt đầu từ ứng dụng khách và kết thúc bằng một quyết định có reason code. Evaluation API không thay đổi cấu hình, không ghi audit log quản trị và không yêu cầu Dashboard tham gia trong runtime.



Hình 2.3: Luồng đánh giá Feature Flag từ ứng dụng khách đến Rule Evaluation Engine

2.4.5 Luồng tích hợp ứng dụng khách

Ứng dụng khách có thể gọi trực tiếp Evaluation API hoặc sử dụng JavaScript SDK. Trong mini project, Demo Application dùng SDK để giảm mã lặp ở phía client và thể hiện cách tích hợp có thể tái sử dụng.

Khởi tạo evaluation context

Ứng dụng tạo context từ các định danh không nhạy cảm như targeting key, user ID mẫu, role và organization ID mẫu. Context chỉ là đầu vào cho engine, không phải bằng chứng phân quyền.

Gọi trực tiếp Evaluation API

Cách gọi trực tiếp phù hợp cho kiểm thử hoặc hệ thống muốn tự quản lý HTTP client. Client gửi project key, environment key, flag key và context, sau đó tự xử lý response.

Gọi thông qua JavaScript SDK

SDK bao bọc cùng contract nhưng bổ sung cấu hình base URL, timeout và fallback. Nội dung triển khai SDK cụ thể được trình bày trong chương kết quả.

Áp dụng kết quả trong ứng dụng

Ứng dụng chỉ sử dụng trường kết quả để bật hoặc ẩn hành vi giao diện tương ứng. Việc này giữ cho logic sản phẩm tách khỏi cách cấu hình flag được quản trị.

Xử lý khi Evaluation API không khả dụng

Khi không nhận được kết quả hợp lệ, client áp dụng mặc định an toàn là tắt tính năng. Quy tắc này tránh việc lỗi hạ tầng vô tình mở tính năng chưa sẵn sàng.

2.4.6 Lựa chọn công nghệ

Các công nghệ được chọn theo tiêu chí phù hợp phạm vi mini project, dễ kiểm thử và đủ gần với một hệ thống thực tế.

Lựa chọn NestJS cho Backend API

NestJS hỗ trợ cấu trúc module rõ ràng, dependency injection, DTO validation và test bằng Jest, phù hợp với backend REST cần tách controller, service và domain logic.

Lựa chọn React cho giao diện web

React phù hợp để xây dựng Dashboard và Demo Application dạng SPA, tái sử dụng component và phản hồi nhanh với trạng thái cấu hình thay đổi.

Lựa chọn PostgreSQL và Prisma

PostgreSQL cung cấp giao dịch và kiểu JSONB cho rule parameters/audit snapshot; Prisma giúp mô hình hóa schema, migration và truy cập dữ liệu có kiểu.

Lựa chọn cache in-memory hoặc Redis

In-memory cache đủ cho demo một instance; Redis là hướng mở rộng khi cần chia sẻ snapshot giữa nhiều backend instance. Thiết kế giữ cache là tùy chọn để không phụ thuộc vào hạ tầng ngoài.

Lựa chọn Docker và Docker Compose

Docker Compose gom backend, frontend, demo và database vào một quy trình khởi chạy thống nhất, thuận tiện cho chấm điểm và trình diễn.

Các phương án thay thế và lý do không lựa chọn

Các lựa chọn như microservice tách riêng evaluation service, message broker hoặc frontend framework khác chưa cần thiết trong phạm vi mini project. Hệ thống ưu tiên kiến trúc đơn giản, có ranh giới rõ và có thể mở rộng sau.

2.5 Thiết kế dữ liệu

Mô hình dữ liệu được thiết kế để lưu cấu hình Feature Flag, phục vụ Evaluation API và ghi nhận lịch sử thay đổi. Nội dung phần này chỉ mô tả cấu trúc lưu trữ; thứ tự đánh giá nằm ở mục Rule Evaluation Engine và contract HTTP nằm ở mục API.

2.5.1 Mô hình dữ liệu tổng thể

Các thực thể chính của hệ thống được tóm tắt trong Bảng 2.4. PostgreSQL được lựa chọn vì hỗ trợ giao dịch, khóa ngoại và kiểu `JSONB` cho rule parameters hoặc audit snapshot [11].

Bảng 2.4: Các thực thể và quan hệ chính trong mô hình dữ liệu

Thực thể	Vai trò chính
`Project`	Phạm vi quản lý cao nhất của một ứng dụng/sản phẩm.
`Environment`	Môi trường triển khai như staging hoặc production.
`FeatureFlag`	Định nghĩa ổn định của flag trong project.
`FlagEnvironmentConfig`	Cấu hình runtime của một flag trong một environment.
`FlagRule`	Rule targeting thuộc một cấu hình runtime.
`FlagGroup`	Nhóm các flag liên quan trong cùng project.
`FlagGroupConfig`	Trạng thái group kill switch theo environment.
`SampleUserContext`	Dữ liệu người dùng mẫu phục vụ demo/kiểm thử.
`AuditLogEntry`	Bản ghi append-only cho mutation quản trị.

Các thực thể chính

Các thực thể được tổ chức theo chuỗi: project chứa environment và flag; flag có cấu hình theo environment; cấu hình có rule; group và audit log bổ sung khả năng vận hành/truy vết.

Quan hệ giữa các thực thể

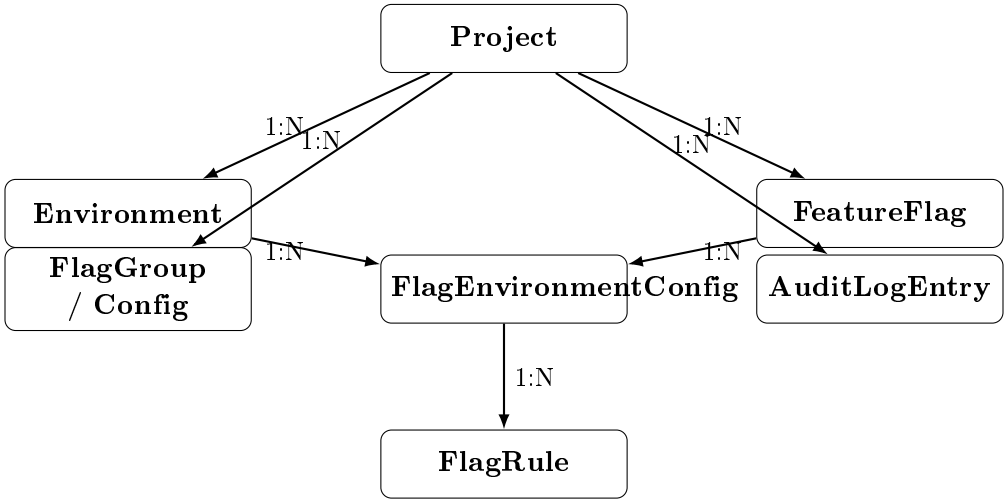
Quan hệ quan trọng nhất là `FeatureFlag` – `FlagEnvironmentConfig` – `Environment`, cho phép một flag có cấu hình khác nhau theo từng môi trường. Group và audit log đều nằm trong phạm vi project để tránh lẫn dữ liệu giữa các ứng dụng.

Phân tách định nghĩa flag và cấu hình runtime

`FeatureFlag` lưu danh tính, tên, mô tả và lifecycle; `FlagEnvironmentConfig` lưu enabled/disabled, serving mode và kill switch theo environment. Việc tách này tránh nhân bản flag khi production và staging cần cấu hình khác nhau.

Sơ đồ thực thể liên kết

Sơ đồ rút gọn trong Hình 2.4 nhấn mạnh đường đi dữ liệu phục vụ evaluation.



Hình 2.4: Sơ đồ thực thể liên kết rút gọn của hệ thống Feature Flag

2.5.2 Project và environment

Project xác định phạm vi nghiệp vụ; environment xác định phạm vi triển khai. Khóa nghiệp vụ của hai thực thể này được dùng trong API, cache key và seed data.

Thực thể Project

`Project` gồm `id`, `key`, `name`, `description`, `createdAt` và `updatedAt`. `key` là duy nhất toàn hệ thống và ổn định trong vòng đời project.

Thực thể Environment

`Environment` gồm `id`, `projectId`, `key`, `name`, `description`, `isDefault` và timestamp. `key` là duy nhất trong phạm vi project.

Environment mặc định

Mỗi project có một environment mặc định để tạo cấu hình ban đầu cho flag mới và giúp demo chạy ngay sau seed.

Ràng buộc environment trong project

Environment luôn thuộc một project cụ thể; các truy vấn evaluation dùng cặp `projectKey`/`environmentKey` để tránh nhầm cấu hình giữa project.

2.5.3 Feature Flag và cấu hình theo environment

Thiết kế tách metadata của flag khỏi trạng thái runtime nhằm hỗ trợ nhiều environment.

Thực thể FeatureFlag

`FeatureFlag` gồm `id`, `projectId`, `key`, `name`, `description`, `status` và timestamp.

Trạng thái vòng đời của flag

Lifecycle status gồm active và archived. Archived giữ lịch sử nhưng không còn phục vụ runtime.

Thực thể FlagEnvironmentConfig

`FlagEnvironmentConfig` nối flag với environment và lưu `enabled`, `servingMode`, `killSwitch` cùng timestamp.

Trạng thái cấu hình và serving mode

Config disabled dừng flag trong environment; serving mode quyết định bật toàn cục hay xét rule khi config enabled.

Cấu hình flag theo từng environment

Cùng một flag có thể bật ở staging, rollout một phần ở production và disabled ở environment khác mà không cần tạo nhiều flag.

2.5.4 Rule và tham số rule

Rule được lưu riêng để có thể thay đổi chiến lược rollout mà không sửa định nghĩa flag.

Thực thể FlagRule

`FlagRule` gồm `id`, `flagConfigId`, `type`, `priority`, `enabled`, `parameters` và timestamp.

Kiểu rule

Mini project hỗ trợ ba kiểu: `USER_ALLOWLIST`, `ROLE_TARGETING` và `PERCENTAGE_ROLLOUT`.

Thứ tự ưu tiên của rule

Database lưu `priority` cho rule cùng loại; type precedence của engine được định nghĩa riêng trong mục evaluation.

Lưu trữ tham số bằng JSON

`parameters` dùng `JSONB` để lưu danh sách user, danh sách role hoặc tỷ lệ rollout mà không cần tạo bảng phụ cho từng kiểu rule.

Validation tham số theo loại rule

Backend kiểm tra shape của `parameters`: user/role phải là danh sách chuỗi, percentage phải nằm trong miền hợp lệ.

2.5.5 User context mẫu

`SampleUserContext` phục vụ demo và kiểm thử, không phải nguồn danh tính bắt buộc của Evaluation API.

Targeting key

`targetingKey` là khóa ổn định cho percentage rollout và là duy nhất trong project demo.

Các trường nghiệp vụ và attributes

Dữ liệu mẫu có thể chứa `userId`, `role` và `attributes` như organization ID.

Phạm vi sử dụng dữ liệu mẫu

Dữ liệu này giúp chọn nhanh context trên Dashboard/Demo Application và không đại diện cho user store sản xuất.

2.5.6 Flag group và group kill switch

Flag group hỗ trợ quản lý các flag liên quan và cho phép ngắt nhanh cả nhóm trong một environment.

Thực thể FlagGroup và FlagGroupConfig

`FlagGroup` lưu định danh nhóm; `FlagGroupConfig` lưu kill switch của nhóm theo environment.

Quan hệ giữa group và flag

Một group có nhiều flag; trong phạm vi mini project, một flag thuộc tối đa một group.

Trạng thái group kill switch theo environment

Kill switch của group được lưu theo environment để tránh ảnh hưởng đồng thời staging và production.

Audit log và cache invalidation cho group

Mutation group tạo audit entry và invalidation snapshot của các flag thành viên bị ảnh hưởng.

2.5.7 Audit log và lịch sử cấu hình

Audit log là nguồn truy vết chính cho mọi mutation quản trị và cũng là cơ sở cho audit-backed history [10].

Cấu trúc AuditLogEntry

`AuditLogEntry` gồm `id`, `projectId`, `actor`, `requestId`, `targetType`, `targetId`, `targetKey`, `action`, `before`, `after` và `createdAt`.

Thông tin actor và request

Actor và request ID được xác định ở backend, không lấy trực tiếp từ body request, để tránh giả mạo lịch sử.

Snapshot trước và sau thay đổi

Snapshot `before`/`after` dùng `JSONB`, đủ để so sánh thay đổi nhưng không lưu bí mật hoặc dữ liệu cá nhân không cần thiết.

Tính chất append-only

Ứng dụng chỉ thêm audit entry mới, không cung cấp luồng sửa hoặc xóa bản ghi đã tạo.

Lịch sử cấu hình dựa trên audit log

History API dựng lại diễn biến cấu hình từ các audit entry liên quan thay vì duy trì bảng version riêng.

2.5.8 Ràng buộc toàn vẹn dữ liệu

Ràng buộc dữ liệu được đặt ở cả Prisma/database và tầng validation để giữ cấu hình hợp lệ trước khi đi vào engine.

Ràng buộc khóa và tính duy nhất

Bảng 2.5: Các ràng buộc duy nhất tiêu biểu

Thực thể	Ràng buộc
`Project`	`key` duy nhất toàn hệ thống.
`Environment`	`projectId, key` duy nhất trong project.
`FeatureFlag`	`projectId, key` duy nhất trong project.
`FlagEnvironmentConfig`	`featureFlagId, environmentId` duy nhất.
`FlagGroupConfig`	`groupId, environmentId` duy nhất.

Ràng buộc quan hệ tham chiếu

Khóa ngoại bảo đảm flag, config, rule, group và audit entry không trỏ đến project hoặc environment không tồn tại.

Ràng buộc giá trị phần trăm rollout

Percentage rollout phải nằm trong miền hợp lệ, thường từ 0 đến 100.

Ràng buộc không sửa hoặc xóa audit log

Audit log không có API update/delete; triển khai production có thể bổ sung trigger database để cưỡng chế thêm.

Chiến lược archive thay cho xóa vật lý

Flag không còn dùng được archive thay vì xóa vật lý để giữ lịch sử và tránh phá vỡ audit/history.

2.6 Thiết kế Rule Evaluation Engine

Rule Evaluation Engine là thành phần quyết định một Feature Flag bật hay tắt cho từng request. Mục này là nơi duy nhất trình bày chi tiết thứ tự đánh giá, cách dùng context, stable hashing, reason code và fallback; các mục dữ liệu/API chỉ tham chiếu đến contract này ở mức tổng quan.

2.6.1 Evaluation request và context

Evaluation request gồm khóa project, environment, flag và một evaluation context do ứng dụng khách cung cấp. Context chỉ là đầu vào cho rollout/targeting, không thay thế cơ chế xác thực hay phân quyền nghiệp vụ.

Doạn mã 2.2: Cấu trúc evaluation request

```
{
  "projectKey": "ecommerce-demo",
  "environmentKey": "production",
  "flagKey": "new-checkout",
  "context": {
    "targetingKey": "user-123",
    "userId": "user-123",
    "role": "BETA_CUSTOMER",
    "attributes": {
      "organizationId": "org-01"
    }
  }
}
```

Project key, environment key và flag key

Ba khóa này xác định chính xác cấu hình cần evaluate. Nếu project hoặc flag không tồn tại, engine trả kết quả tắt an toàn với reason `NOT_FOUND`.

Targeting key

`targetingKey` là định danh ổn định, không chứa PII, dùng cho percentage rollout. Khi thiếu trường này, rule percentage không đủ điều kiện để match.

User ID, role và organization ID

`userId` phục vụ allowlist; `role` phục vụ role targeting; `attributes.organizationId` được giữ như dữ liệu mở rộng cho demo và hướng phát triển sau này.

Thuộc tính mở rộng

`attributes` cho phép bổ sung dữ liệu mà không đổi contract chính. Engine hiện chỉ dùng một phần nhỏ, các rule theo thuộc tính nâng cao được để ở hướng phát triển.

Validation evaluation request

Backend kiểm tra kiểu dữ liệu và định dạng bắt buộc trước khi gọi engine. Request sai định dạng bị từ chối ở biên API, còn request hợp lệ nhưng thiếu dữ liệu cho một rule sẽ được engine xử lý theo hướng bỏ qua rule đó.

2.6.2 Evaluation snapshot

Snapshot là ảnh chụp cấu hình cần thiết tại thời điểm evaluate, gồm trạng thái flag, config theo environment, group config và danh sách rule đã chuẩn hóa.

Khái niệm evaluation snapshot

Snapshot giúp engine không phụ thuộc trực tiếp vào ORM hoặc controller. Cùng một snapshot và cùng một context phải luôn tạo ra cùng một kết quả.

Dữ liệu cấu hình cần truy xuất

Snapshot chứa lifecycle flag, trạng thái enabled/disabled, serving mode, kill switch, group kill switch và rule parameters. Dữ liệu audit/history không nằm trong snapshot runtime.

Sắp xếp rule trước khi evaluation

Repository chuẩn bị rule theo loại và priority để engine chỉ tập trung vào logic quyết định. Cách sắp xếp này làm kết quả ổn định giữa các lần gọi.

Tính nhất quán của snapshot

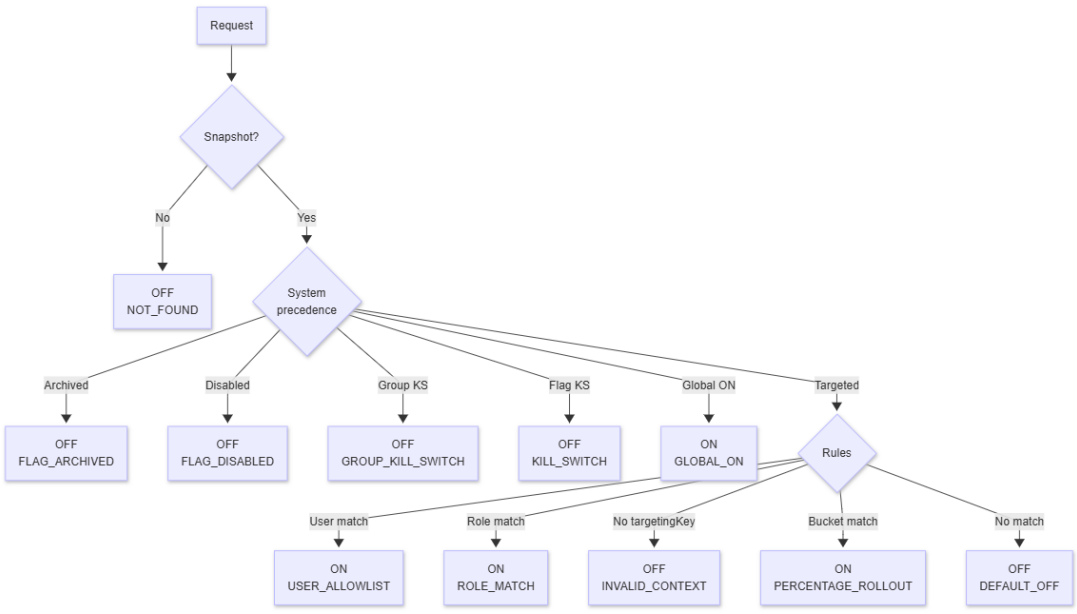
Snapshot được lấy từ database hoặc cache đã invalidation sau mutation. Nếu cache miss, backend nạp lại snapshot từ nguồn dữ liệu chính.

2.6.3 Thứ tự ưu tiên khi đánh giá

Thứ tự ưu tiên là hợp đồng quan trọng nhất của engine. Các điều kiện an toàn được xét trước rule rollout để tránh việc một flag bị tắt nhưng vẫn vô tình được bật bởi rule phía sau.

Đoạn mã 2.3: Thứ tự ưu tiên của Rule Evaluation Engine

```
ARCHIVED_FLAG
FLAG_DISABLED
GROUP_KILL_SWITCH
KILL_SWITCH
GLOBAL_ON
USER_ALLOWLIST
ROLE_TARGETING
PERCENTAGE_ROLLOUT
DEFAULT_OFF
```



Hình 2.5: Flowchart thứ tự ưu tiên của Rule Evaluation Engine

Flag đã được archive

Flag archived không tham gia phục vụ runtime, vì vậy engine trả tắt trước mọi cấu hình khác.

Flag bị disabled

Nếu config theo environment đang disabled, engine trả tắt ngay cả khi còn rule đang enabled.

Group kill switch

Khi group kill switch bật cho environment hiện tại, toàn bộ flag thuộc group bị tắt sau khi đã qua kiểm tra archived/disabled.

Flag kill switch

Flag-level kill switch là lớp dừng khẩn cấp cho một flag riêng lẻ.

Global on

Nếu config enabled và serving mode là global on, engine bật flag cho mọi context hợp lệ mà không cần xét rule targeting.

User allowlist

Allowlist được xét trước các rule mở rộng phạm vi vì đây là điều kiện cụ thể nhất.

Role targeting

Role targeting được xét sau allowlist để hỗ trợ phát hành theo nhóm người dùng.

Percentage rollout

Percentage rollout là lớp cuối trong nhóm rule vì nó mở rộng phát hành theo tỷ lệ.

Default off

Nếu không có điều kiện nào match, engine trả tắt an toàn.

2.6.4 Đánh giá trạng thái toàn cục

Trạng thái lifecycle, enabled/disabled, serving mode và kill switch quyết định các nhánh đầu của engine. Bảng 2.6 tóm tắt các trường hợp chính.

Bảng 2.6: Tương tác giữa trạng thái và serving mode

Lifecycle/config	Kill switch	Serving mode	Kết quả ưu tiên
Archived	Bất kỳ	Bất kỳ	Tắt, `FLAG_ARCHIVED`
Disabled	Bất kỳ	Bất kỳ	Tắt, `FLAG_DISABLED`
Enabled	Group on	Bất kỳ	Tắt, `GROUP_KILL_SWITCH`
Enabled	Flag on	Bất kỳ	Tắt, `FLAG_KILL_SWITCH`
Enabled	Off	Global on	Bật, `GLOBAL_ON`
Enabled	Off	Targeted	Xét rule theo precedence

2.6.5 Targeting theo người dùng

User targeting dựa trên `context.userId`. Rule chỉ match khi user ID trong context thuộc danh sách đã cấu hình.

User allowlist

Allowlist phù hợp cho kiểm thử nội bộ hoặc mở tính năng cho một nhóm nhỏ đã biết trước.

So khớp user ID

Việc so khớp dùng giá trị chính xác, không suy diễn từ targeting key để tránh nhầm giữa định danh rollout và định danh nghiệp vụ.

Tách userId và targetingKey

`userId` dùng cho allowlist; `targetingKey` dùng cho hashing. Tách hai trường giúp hệ thống dùng một khóa rollout không chứa PII.

Xử lý khi thiếu định danh người dùng

Nếu thiếu `userId`, allowlist không match nhưng engine vẫn có thể xét role hoặc percentage nếu context còn đủ dữ liệu cho các rule đó.

2.6.6 Targeting theo vai trò

Role targeting dựa trên vai trò do context cung cấp và phù hợp với demo phân nhóm người dùng.

Cấu trúc role trong evaluation context

Role có thể là một chuỗi hoặc danh sách chuỗi tùy contract triển khai; backend chuẩn hóa trước khi so khớp.

So khớp một hoặc nhiều role

Rule match khi ít nhất một role của context nằm trong danh sách role được cấu hình.

Quan hệ giữa role targeting và user targeting

Allowlist có precedence cao hơn role targeting, vì một người dùng được chỉ định trực tiếp nên được quyết định trước nhóm vai trò rộng hơn.

Xử lý khi context không chứa role

Nếu thiếu role, các rule role targeting bị bỏ qua và engine tiếp tục xét lớp rule tiếp theo.

2.6.7 Rollout theo phần trăm người dùng

Percentage rollout dùng stable hashing để phân bổ một targeting key vào bucket cố định từ 0 đến 99.

Mục tiêu của percentage rollout

Mục tiêu là phát hành dần tính năng cho một tỷ lệ người dùng ổn định, tránh thay đổi kết quả ngẫu nhiên giữa các request.

Stable hashing

Hệ thống băm tổ hợp project/environment/flag/targeting key để cùng một người dùng giữ kết quả ổn định trong phạm vi flag và environment.

Đoạn mã 2.4: Tạo giá trị băm ổn định bằng SHA-256

```
const hashInput = `${projectKey}:${environmentKey}:${flagKey}:${targetingKey
const digest = sha256(hashInput);
```

Thành phần đầu vào của hàm băm

Đầu vào băm bao gồm các khóa cấu hình và targeting key. Không sử dụng thời gian hiện tại hoặc số ngẫu nhiên để đảm bảo tính xác định.

Chuyển đổi kết quả băm thành bucket

Một phần digest được chuyển thành số nguyên rồi lấy modulo 100 để tạo bucket phần trăm.

Đoạn mã 2.5: Chuyển đổi SHA-256 digest thành bucket phần trăm

```
const bucket = parseInt(digest.slice(0, 8), 16) % 100;
const enabled = bucket < percentage;
```

Persistent assignment

Nếu input không đổi, bucket không đổi; vì vậy người dùng không bị nhảy qua lại giữa enabled và disabled.

Ảnh hưởng khi thay đổi targeting key

Thay đổi targeting key làm thay đổi bucket, do đó ứng dụng cần chọn khóa ổn định ngay từ đầu.

2.6.8 Thực thi rule theo độ ưu tiên

Rule chỉ được thực thi sau khi các điều kiện hệ thống đã cho phép flag tham gia evaluation.

Sắp xếp rule

Trong từng loại rule, các rule enabled được xét theo priority tăng dần.

Type precedence và first match

Engine áp dụng type precedence cố định: allowlist, role targeting, percentage rollout. Khi một rule match, engine trả kết quả ngay.

Rule không hợp lệ hoặc không đủ context

Rule sai tham số hoặc thiếu dữ liệu context bị bỏ qua thay vì làm bật tính năng ngoài ý muốn.

2.6.9 Variant, EvaluationResult và reason code

Kết quả evaluation phải vừa đủ cho ứng dụng dùng, Dashboard/debug hiểu được, nhưng không phơi bày toàn bộ cấu hình quản trị.

Variant và response shape

Đoạn mã 2.6: Cấu trúc EvaluationResult

```
{
  "enabled": true,
  "variant": "on",
  "reason": "USER_ALLOWLIST",
  "projectKey": "ecommerce-demo",
  "flagKey": "new-checkout"
}
```

Tập reason code

Đoạn mã 2.7: Tập reason code của Evaluation API

```
NOT_FOUND
FLAG_ARCHIVED
FLAG_DISABLED
GROUP_KILL_SWITCH
KILL_SWITCH
GLOBAL_ON
USER_ALLOWLIST
ROLE_MATCH
PERCENTAGE_ROLLOUT
DEFAULT_OFF
INVALID_CONTEXT
ERROR
```

Bảng 2.7: Reason code cho trạng thái hệ thống

Reason	Ý nghĩa
NOT_FOUND	Không tìm thấy project hoặc flag.
FLAG_ARCHIVED	Flag không còn phục vụ runtime.
FLAG_DISABLED	Config theo environment đang disabled.
GROUP_KILL_SWITCH	Group kill switch đang bật.
FLAG_KILL_SWITCH	Kill switch riêng của flag đang bật.
ERROR	Lỗi không mong muốn, trả fail-closed.

Bảng 2.8: Quan hệ giữa mã lý do và matched rule

Reason	Matched rule
USER_TARGET_MATCH	Rule allowlist match `userId`.
ROLE_TARGET_MATCH	Rule role targeting match role của context.
PERCENTAGE_ROLLOUT_MATCH	Bucket stable hashing nằm trong tỷ lệ rollout.
DEFAULT_OFF	Không rule nào match.

Metadata phục vụ truy vết

Response có thể kèm metadata như matched rule ID hoặc cache status khi cần debug, nhưng metadata này không bắt buộc đối với ứng dụng khách.

2.6.10 Default off và error fallback

Nguyên tắc chung là fail-closed: khi thiếu thông tin hoặc gặp lỗi, hệ thống ưu tiên tắt tính năng mới.

Nguyên tắc fail-safe và fail-closed

Fail-closed giúp lỗi hạ tầng không trở thành hành vi mở tính năng ngoài kế hoạch.

Đoạn mã 2.8: Kết quả fallback mặc định

```
{
  "enabled": false,
  "variant": "off",
  "reason": "ERROR"
}
```

Flag không tồn tại

Không tìm thấy project hoặc flag trả `enabled=false` và `reason=NOT_FOUND`.

Context không đầy đủ

Context thiếu dữ liệu chỉ làm rule tương ứng không match; nếu không còn rule nào match thì kết quả cuối là `DEFAULT_OFF` hoặc `INVALID_CONTEXT` tùy loại lỗi.

Lỗi database hoặc cache

Khi không tải được snapshot, backend không suy đoán kết quả mà trả fallback tắt.

Lỗi nội bộ của Evaluation Engine

Đoạn mã 2.9: Xử lý lỗi nội bộ của Evaluation Engine

```
try {
  return evaluateWithSnapshot(snapshot, context);
} catch (error) {
  return disabled("ERROR");
}
```

2.6.11 Khả năng mở rộng Rule Evaluation Engine

Engine được thiết kế theo hướng thêm loại rule mới mà không thay đổi contract response. Các hướng mở rộng phù hợp gồm targeting theo thuộc tính, organization, thời gian, vị trí và phân phối nhiều variant cho A/B testing.

2.7 Thiết kế API

2.7.1 Nguyên tắc thiết kế API

API của hệ thống được tổ chức theo kiến trúc REST, sử dụng JSON, tiền tố phiên bản `/v1` và quy ước trường dữ liệu dạng `camelCase`. Thiết kế hiện tại tách rõ hai nhóm endpoint:

- **Control plane**: các endpoint quản trị project, feature flag, rule, group, sample user, audit log, lịch sử cấu hình và thống kê. Nhóm này yêu cầu bearer token demo và được kiểm tra quyền ở backend.

- **Data plane**: endpoint `POST /v1/evaluate`. Endpoint này phục vụ runtime evaluation cho demo app và SDK, không yêu cầu bearer token quản trị.

Các endpoint quản trị dùng khóa nghiệp vụ ổn định như `projectKey`, `flagKey` và `groupKey` trên URL thay vì yêu cầu frontend phải biết định danh nội bộ. Cách này khớp với source code hiện tại trong các controller NestJS và thuận tiện cho demo, vì người trình bày có thể đọc URL theo đúng ngữ nghĩa nghiệp vụ.

Versioning đường dẫn API

Toàn bộ REST API được đặt dưới tiền tố `/v1`. Một số endpoint chính trong source code hiện tại gồm:

```
POST /v1/evaluate
GET  /v1/projects
GET  /v1/projects/{projectKey}/flags
GET  /v1/projects/{projectKey}/groups
GET  /v1/projects/{projectKey}/audit-logs
GET  /v1/projects/{projectKey}/stats/flags
```

Trong phạm vi mini project, chỉ có phiên bản `v1`. Nếu sau này cần thay đổi không tương thích ngược về request hoặc response, hệ thống có thể bổ sung `/v2` mà không phá vỡ client đang dùng `/v1`.

Phân trang, lọc và sắp xếp

Các endpoint trả về danh sách sử dụng offset pagination với `limit` và `offset`. Giá trị mặc định là `limit = 20`, `offset = 0`; `limit` không vượt quá 100. Response danh sách trong backend hiện tại có dạng:

```
{
  "items": [],
  "page": {
    "limit": 20,
    "offset": 0,
    "total": 42,
    "hasNext": true
```

```
  }
}
```

Các query filter phụ thuộc từng endpoint. Ví dụ, danh sách flag hỗ trợ `search`, `status` và `lifecycleStatus`; rule hỗ trợ lọc theo `type`; audit log hỗ trợ lọc theo `targetType`, `targetKey`, `actor`, `action`, `from` và `to`; statistics hỗ trợ `environmentKey`, khoảng thời gian và sort theo tổng số lượt evaluation.

2.7.2 API quản lý project và environment

Project là phạm vi quản lý cấp cao nhất. Source code hiện tại triển khai các endpoint project sau:

- `GET /v1/projects`: danh sách project;
- `POST /v1/projects`: tạo project;
- `GET /v1/projects/{projectKey}`: xem chi tiết project;
- `PATCH /v1/projects/{projectKey}`: cập nhật metadata;
- `DELETE /v1/projects/{projectKey}`: soft delete project.

Endpoint quản lý environment riêng chưa được mở ra trong phiên bản hiện tại. Environment vẫn là một phần của mô hình dữ liệu và seed data, gồm các môi trường như `production`, `staging` và `development`. Evaluation API có thể nhận `environmentKey`; nếu client không truyền trường này, backend dùng environment mặc định của project. Nhờ đó, hệ thống vẫn chứng minh được khả năng cấu hình theo môi trường mà không làm dashboard phức tạp quá mức.

2.7.3 API quản lý Feature Flag

Feature Flag được quản lý theo phạm vi project và được định danh bằng `flagKey`. Các endpoint hiện tại gồm:

- `GET /v1/projects/{projectKey}/flags`: danh sách flag đang hoạt động;
- `GET /v1/projects/{projectKey}/flags/deleted`: danh sách flag đã soft delete;
- `POST /v1/projects/{projectKey}/flags`: tạo flag;
- `GET /v1/projects/{projectKey}/flags/{flagKey}`: xem chi tiết flag;

- `PATCH /v1/projects/{projectKey}/flags/{flagKey}`: cập nhật metadata và cấu hình runtime của default environment;
- `POST /v1/projects/{projectKey}/flags/{flagKey}/archive`: archive flag;
- `POST /v1/projects/{projectKey}/flags/{flagKey}/restore`: restore flag đã archive;
- `DELETE /v1/projects/{projectKey}/flags/{flagKey}`: soft delete flag;
- `POST /v1/projects/{projectKey}/flags/{flagKey}/restore-deleted`: khôi phục flag đã soft delete.

Khi tạo flag, backend tạo cấu hình `FlagEnvironmentConfig` mặc định với `status = DISABLED`, `servingMode = TARGETED` và `killSwitch = false`. Khi cập nhật `status`, `servingMode` hoặc `killSwitch`, backend ghi audit log trong cùng transaction và invalidation evaluation snapshot cache của flag liên quan.

Cần phân biệt `lifecycleStatus` của flag với trạng thái runtime. `ACTIVE` hoặc `ARCHIVED` mô tả vòng đời của flag; còn `enabled` trong Evaluation API là kết quả On/Off tại runtime.

2.7.4 API quản lý rule

Source code hiện tại không dùng endpoint CRUD từng rule riêng lẻ. Dashboard gửi toàn bộ danh sách rule mong muốn cho một flag bằng endpoint replace:

- `GET /v1/projects/{projectKey}/flags/{flagKey}/rules`: xem rule hiện tại;
- `PUT /v1/projects/{projectKey}/flags/{flagKey}/rules`: thay thế toàn bộ ordered rule set.

Request `PUT` chứa mảng `rules`; mỗi rule có `type`, `priority`, `enabled` và `parameters`. Ba loại rule được hỗ trợ là `USER_ALLOWLIST`, `ROLE_TARGETING` và `PERCENTAGE_ROLLOUT`. Với source code hiện tại, user allowlist dùng `parameters.userIds`; role targeting dùng `parameters.roles`; percentage rollout dùng `parameters.percentage`. Rule bị disable được bỏ qua khi evaluation.

Việc replace toàn bộ danh sách rule giúp UI dễ lưu thứ tự hiện tại và giúp audit log ghi được snapshot trước/sau của toàn bộ rule set bằng action `FLAG_RULES_REPLACED`.

2.7.5 API quản lý flag group và group kill switch

Flag group là chức năng khuyến khích đã được triển khai để hỗ trợ kill switch theo nhóm. Các endpoint chính gồm:

- `GET /v1/projects/{projectKey}/groups`: danh sách group;
- `POST /v1/projects/{projectKey}/groups`: tạo group;
- `PATCH /v1/projects/{projectKey}/groups/{groupKey}`: đổi tên group;
- `DELETE /v1/projects/{projectKey}/groups/{groupKey}`: xóa group;
- `PUT /v1/projects/{projectKey}/groups/{groupKey}/config`: bật hoặc tắt kill switch của group trong một environment;
- `PUT /v1/projects/{projectKey}/flags/{flagKey}/group`: gán flag vào group;
- `DELETE /v1/projects/{projectKey}/flags/{flagKey}/group`: gỡ flag khỏi group.

Trạng thái kill switch của group được lưu trong `FlagGroupConfig` theo từng environment. Khi group kill switch bật, Evaluation Engine trả `enabled = false` với reason `GROUP_KILL_SWITCH` sau khi đã kiểm tra `FLAG_ARCHIVED` và `FLAG_DISABLED`, nhưng trước flag-level `KILL_SWITCH`, `GLOBAL_ON` và các rule targeting.

2.7.6 Evaluation API

Evaluation API là data-plane endpoint công khai của demo:

```
POST /v1/evaluate
```

Request body gồm project, flag, environment tùy chọn và evaluation context:

```
{
  "projectKey": "demo-project",
  "environmentKey": "production",
  "flagKey": "new-checkout",
  "context": {
    "targetingKey": "demo-user-beta",
    "userId": "demo-user-beta",
    "roles": ["beta-customer"]
```

```
  }
}
```

Response luôn chứa `projectKey`, `flagKey`, `enabled`, `variant`, `reason` và `matchedRuleId`:

```
{
  "projectKey": "demo-project",
  "flagKey": "new-checkout",
  "enabled": true,
  "variant": "on",
  "reason": "ROLE_MATCH",
  "matchedRuleId": "rule-id"
}
```

Nếu project, environment, flag hoặc flag environment config không tồn tại, API trả evaluation-shaped response an toàn với `enabled = false`, `variant = off`, `reason = NOT_FOUND`. Nếu percentage rollout được đánh giá nhưng thiếu `targetingKey`, API trả `INVALID_CONTEXT`. Lỗi bất ngờ trong evaluation path được bắt lại và trả `ERROR` theo nguyên tắc fail-closed.

Các reason code hiện tại gồm `GLOBAL_ON`, `FLAG_DISABLED`, `FLAG_ARCHIVED`, `GROUP_KILL_SWITCH`, `KILL_SWITCH`, `USER_ALLOWLIST`, `ROLE_MATCH`, `PERCENTAGE_ROLLOUT`, `DEFAULT_OFF`, `NOT_FOUND`, `INVALID_CONTEXT` và `ERROR`.

2.7.7 API audit log và lịch sử cấu hình

Audit log được truy vấn trong phạm vi project:

- `GET /v1/projects/{projectKey}/audit-logs`: danh sách audit log;
- `GET /v1/projects/{projectKey}/flags/{flagKey}/history`: lịch sử cấu hình tập trung của một flag.

Audit log lưu `actor`, `action`, `targetType`, `targetId`, `targetKey`, `before`, `after`, `metadata`, `requestId` và thời điểm tạo. Các thao tác mutation quan trọng như tạo/cập nhật project, tạo/cập nhật/archive/restore/delete flag, replace rule, tạo/cập nhật group, bật/tắt group kill switch và gán/gỡ group đều ghi audit log cùng transaction với thay đổi nghiệp vụ.

Endpoint history không tạo bảng version riêng. Nó là projection đọc từ `AuditLogEntry` cho

flag, flag environment config và rule liên quan. Cách này đáp ứng yêu cầu lịch sử cấu hình mà không tạo hai nguồn sự thật song song.

2.7.8 API thống kê evaluation

Thống kê evaluation là chức năng khuyến khích đã được triển khai bằng dữ liệu aggregate theo giờ UTC, không lưu raw evaluation context. Các endpoint hiện tại gồm:

- `GET /v1/projects/{projectKey}/stats/flags`: tổng hợp theo các flag trong project;
- `GET /v1/projects/{projectKey}/flags/{flagKey}/stats`: thống kê chi tiết một flag.

Dữ liệu thống kê được nhóm theo `projectKey`, `environmentKey`, `flagKey`, `bucketStart`, `reason` và `enabled`. Hệ thống cố ý không lưu targeting key, user ID, role, attributes hay matched rule ID trong bảng metric. Việc ghi metric là best-effort; nếu ghi metric thất bại, Evaluation API vẫn trả kết quả cho client.

2.7.9 Validation và mô hình lỗi

Backend sử dụng DTO, `class-validator`, `ValidationPipe` và global exception filter để chuẩn hóa validation và error response. Key nghiệp vụ tuân theo biểu thức chính quy:

```
^[a-z0-9][a-z0-9-]{1,62}[a-z0-9]$
```

Điều này tương ứng với độ dài 3–64 ký tự, chỉ gồm chữ thường, chữ số và dấu gạch ngang, không bắt đầu hoặc kết thúc bằng dấu gạch ngang. Error response control-plane có dạng:

```
{
  "code": "VALIDATION_ERROR",
  "message": "Request validation failed.",
  "details": [
    { "field": "projectKey", "message": "Invalid key format" }
  ],
  "requestId": "request-id"
}
```

Các mã lỗi hiện tại gồm `VALIDATION_ERROR`, `UNAUTHORIZED`, `FORBIDDEN`, `NOT_FOUND`, `CONFLICT` và `INTERNAL_ERROR`. Request ID được lấy từ header `X-Request-Id` nếu có, hoặc được backend

tạo tự động; giá trị này xuất hiện trong error response, audit log và log phía server để hỗ trợ truy vết [9, 10].

2.7.10 Tài liệu hóa API

Backend sinh tài liệu OpenAPI/Swagger từ controller và DTO. Sau khi backend khởi động, Swagger UI có tại:

```
/docs
```

Swagger được cấu hình với bearer auth cho control-plane demo token và API key `X-Request-Id`. Các tag chính gồm Health, Projects, Feature Flags, Rules, Evaluation, Statistics, Sample Users và Audit Logs. Việc sinh tài liệu từ source code giúp báo cáo, frontend, SDK và kiểm thử bám sát hợp đồng triển khai thực tế.

2.8 Hiệu năng, nhất quán và bảo mật

Mục này tổng hợp các quyết định phi chức năng của hệ thống. Nội dung chỉ tập trung vào nguyên tắc thiết kế; cơ chế triển khai cụ thể của cache, RBAC và statistics được trình bày trong chương kết quả.

2.8.1 Caching evaluation snapshot

Cache được dùng để giảm số lần đọc cấu hình lặp lại khi Evaluation API được gọi thường xuyên. Hệ thống cache snapshot cấu hình thay vì cache final decision theo người dùng, vì quyết định cuối cùng còn phụ thuộc vào evaluation context của từng request.

Mục tiêu của caching

Mục tiêu chính là giảm độ trễ và giảm tải database cho các flag được evaluate nhiều lần. Cache không được làm thay đổi thứ tự ưu tiên hoặc kết quả xác định của engine.

Cache configuration snapshot

Snapshot bao gồm các cấu hình cần thiết để engine ra quyết định cho một flag trong một environment. Khi cache miss, snapshot được nạp lại từ PostgreSQL.

Cache kết quả evaluation

Hệ thống không ưu tiên cache kết quả cuối theo từng context để tránh lưu dữ liệu người dùng và tránh sai lệch khi context thay đổi. Quyết định này đánh đổi một phần hiệu năng để giữ mô hình đơn giản và an toàn hơn.

Lựa chọn in-memory cache và Redis

In-memory cache phù hợp cho demo và kiểm thử cục bộ. Redis được xem là lựa chọn mở rộng khi nhiều backend instance cần dùng chung cache hoặc invalidation phân tán.

2.8.2 Thiết kế cache key và thời gian tồn tại

Cache key phải đủ cụ thể để phân biệt project, environment và flag, nhưng không chứa dữ liệu cá nhân từ evaluation context.

Thành phần của cache key

Khóa cache sử dụng các định danh ổn định của project, environment và flag. Đây là dữ liệu cấu hình, không phải dữ liệu nhận dạng người dùng cuối.

Cache key theo environment

Mỗi environment có cấu hình riêng nên cache key luôn bao gồm environment key. Điều này ngăn việc cấu hình staging bị dùng nhầm cho production.

Cache key theo evaluation context

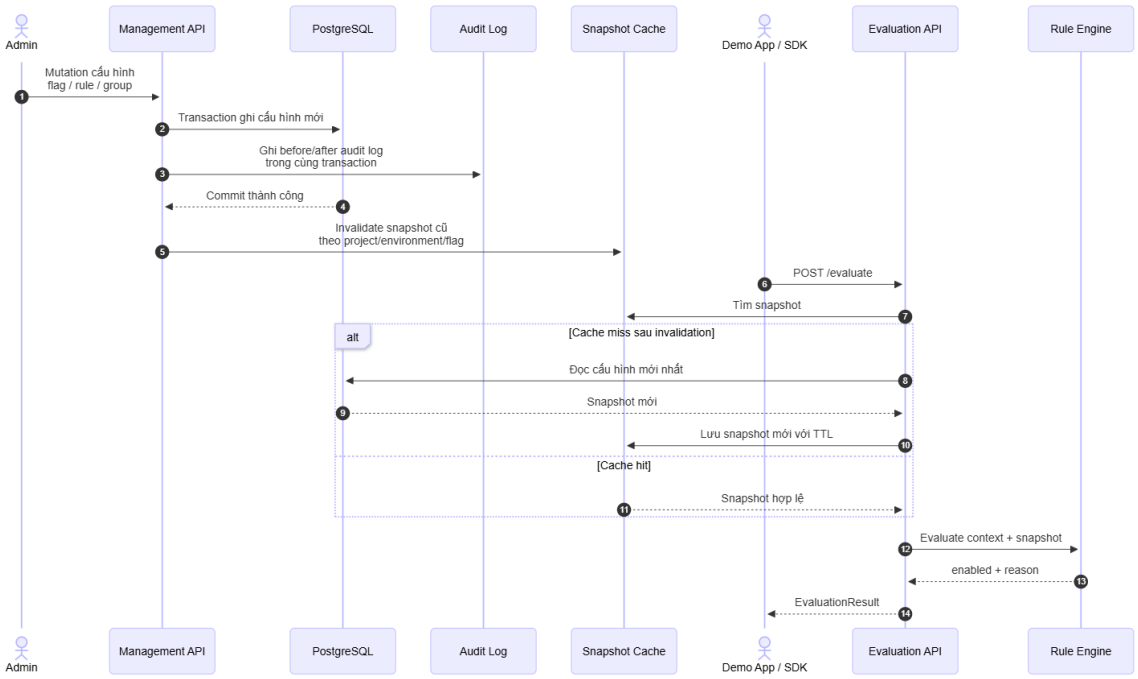
Context không được đưa vào key snapshot. Nếu sau này cache final decision, hệ thống cần thiết kế khóa riêng và đánh giá kỹ rủi ro lưu định danh người dùng.

Lựa chọn thời gian tồn tại

TTL đóng vai trò lớp bảo vệ phụ trong trường hợp invalidation bị bỏ sót. TTL không thay thế invalidation sau mutation cấu hình.

2.8.3 Cache invalidation

Invalidation được kích hoạt khi cấu hình có thể ảnh hưởng đến kết quả evaluation. Nguyên tắc là ưu tiên loại bỏ snapshot hơi rộng còn hơn giữ snapshot lỗi thời.



Hình 2.6: Luồng cache invalidation và bảo đảm tính nhất quán cấu hình

Invalidation khi cập nhật flag

Thay đổi lifecycle, serving mode hoặc kill switch của flag làm mất hiệu lực snapshot tương ứng.

Invalidation khi cập nhật rule

Tạo, cập nhật, xóa mềm hoặc thay đổi priority của rule đều yêu cầu làm mới snapshot của flag chứa rule đó.

Invalidation khi thay đổi group kill switch

Khi group kill switch thay đổi, các snapshot của flag thuộc group trong environment liên quan cần được loại bỏ.

Invalidation theo environment

Invalidation luôn giới hạn theo environment để tránh ảnh hưởng không cần thiết đến cấu hình của môi trường khác.

2.8.4 Tính nhất quán của cấu hình

Hệ thống ưu tiên tính đúng đắn của cấu hình hơn việc giữ cache lâu. Mutation được ghi vào database trước, sau đó cache bị invalidation để data plane đọc lại snapshot mới.

Strong consistency và eventual consistency

Trong phạm vi một backend instance, hệ thống hướng tới việc phản ánh thay đổi gần như tức thời sau mutation. Khi mở rộng nhiều instance, Redis hoặc cơ chế pub/sub sẽ cần thiết để giảm độ trễ đồng bộ.

Stale configuration

Stale configuration là rủi ro chính của caching. Hệ thống giảm rủi ro bằng invalidation theo mutation, TTL và fallback đọc lại từ database khi cache miss.

Giao dịch khi cập nhật cấu hình

Các mutation quan trọng cần lưu thay đổi cấu hình và audit log trong cùng một transaction. Cách này tránh trạng thái có cấu hình mới nhưng thiếu bằng chứng truy vết.

Cân bằng giữa hiệu năng và độ mới dữ liệu

Thiết kế chỉ cache snapshot, không cache mọi response, nhằm giữ latency thấp mà vẫn duy trì khả năng cập nhật cấu hình nhanh.

2.8.5 Trust boundary của evaluation context

Evaluation context do ứng dụng khách gửi lên nên phải được xem là dữ liệu không tin cậy. Engine có thể dùng context để quyết định rollout, nhưng không nên dùng context thay cho xác thực hay phân quyền nghiệp vụ.

Nguồn tạo evaluation context

Context có thể được tạo bởi Demo Application, backend nghiệp vụ hoặc SDK. Nguồn nào tạo context thì nguồn đó chịu trách nhiệm cung cấp định danh ổn định và không nhạy cảm.

Dữ liệu do client tự khai báo

Các trường như role hoặc organization ID trong demo có giá trị minh họa. Trong hệ thống sản xuất, những trường này nên được xác nhận từ server hoặc hệ thống định danh đáng tin cậy.

Nguy cơ giả mạo user ID và role

Nếu client tự khai báo role, người dùng có thể giả mạo để nhận tính năng ngoài ý muốn. Vì vậy Feature Flag không được dùng như cơ chế bảo vệ quyền truy cập duy nhất.

Evaluation phía client và phía server

Client-side evaluation thuận tiện cho UI, còn server-side evaluation an toàn hơn cho chức năng nhạy cảm. Mini project dùng API tập trung để giữ logic đánh giá ở backend.

Giới hạn sử dụng Feature Flag cho phân quyền

Feature Flag chỉ quyết định việc phát hành tính năng, không thay thế authorization. Các endpoint quan trọng vẫn cần kiểm tra quyền ở backend.

2.8.6 Bảo vệ Management API

Management API thay đổi cấu hình hệ thống nên cần được bảo vệ chặt hơn Evaluation API. Các nguyên tắc áp dụng gồm xác thực actor, kiểm tra quyền, validation DTO và audit log.

Xác thực người dùng quản trị

Trong demo, actor được resolver phía server theo bearer token mẫu. Trong production, lớp này cần thay bằng cơ chế định danh thật như OAuth2/OIDC hoặc SSO.

Bảo vệ mutation endpoint

Các endpoint tạo, sửa, archive, restore, thay đổi rule và kill switch phải kiểm tra quyền trước khi ghi dữ liệu.

Validation actor

Actor ghi vào audit log phải đến từ backend sau khi xác thực, không lấy trực tiếp từ body request để tránh giả mạo lịch sử.

Giới hạn truy cập audit log

Audit log có thể chứa snapshot cấu hình nên chỉ các vai trò phù hợp mới được xem hoặc lọc dữ liệu này.

2.8.7 Phân quyền quản trị

RBAC demo phân biệt admin, developer và viewer để thể hiện cách hệ thống kiểm soát thao tác quản trị.

Quyền của admin

Admin có toàn quyền quản lý cấu hình, group, audit log và thống kê trong phạm vi demo.

Quyền của developer

Developer được phép thực hiện các thao tác cấu hình thường gặp nhưng có thể bị giới hạn ở một số thao tác nhạy cảm tùy permission matrix.

Quyền của viewer

Viewer chỉ xem dữ liệu và không được thực hiện mutation.

Kiểm tra quyền tại Backend API

Backend là nơi quyết định quyền cuối cùng. Dashboard chỉ phản ánh trạng thái quyền để cải thiện trải nghiệm, không phải lớp bảo vệ chính.

Hiển thị quyền trên Dashboard

Dashboard ẩn hoặc vô hiệu hóa thao tác không được phép, đồng thời hiển thị role hiện tại để người dùng hiểu phạm vi thao tác.

2.8.8 Auditability và truy vết thay đổi

Auditability được thiết kế như một yêu cầu xuyên suốt: mỗi mutation quan trọng phải có actor, action, target, request ID và snapshot trước/sau. Dữ liệu này hỗ trợ kiểm thử, trình diễn, phân tích sự cố và mở rộng sang các quy trình phê duyệt trong tương lai [10].

2.9 Phương pháp kiểm thử và đánh giá

Phần này chỉ xác định chiến lược kiểm thử dùng cho mini project. Các kết quả kiểm thử cụ thể được trình bày ở Chương 3, còn logic chi tiết của Rule Evaluation Engine, API, cache, RBAC và audit log đã được mô tả trong các phần thiết kế tương ứng.

2.9.1 Phạm vi kiểm thử

Hệ thống được kiểm thử theo bốn lớp chính:

- **Domain và Rule Evaluation Engine**: kiểm tra precedence, targeting rule, percentage rollout, kill switch, default off và fallback.
- **Backend API**: kiểm tra contract của Management API và Evaluation API, bao gồm validation, lỗi không tìm thấy, xung đột dữ liệu và response shape.
- **Control-plane side effects**: kiểm tra audit log, cache invalidation, thống kê evaluation và phân quyền demo.
- **Workflow trình diễn**: kiểm tra Dashboard, Demo Application, JavaScript SDK và Docker Compose trên các kịch bản có thể trình bày trực tiếp.

Cách chia này giúp tránh kiểm thử trùng lặp cùng một hành vi ở nhiều tầng. Unit test tập trung vào quyết định thuần của engine; integration test xác nhận API, persistence và side effect; kịch bản demo xác nhận trải nghiệm end-to-end.

2.9.2 Tiêu chí đánh giá Rule Evaluation Engine

Rule Evaluation Engine được đánh giá theo ba tiêu chí:

- **Đúng precedence**: trạng thái archive, disabled, group kill switch, flag kill switch và global on phải có thứ tự ưu tiên cao hơn các targeting rule.
- **Xác định**: cùng một snapshot cấu hình và cùng một evaluation context phải luôn tạo ra cùng một kết quả, đặc biệt đối với percentage rollout.
- **Fail-closed**: khi flag không tồn tại, context không đủ hoặc engine gặp lỗi, hệ thống không được tự động bật tính năng ngoài ý muốn.

Các test không lặp lại mô tả thuật toán; chúng chỉ tạo dữ liệu đầu vào, gọi engine hoặc Evaluation API và so sánh với kết quả kỳ vọng gồm `enabled`, `variant`, `reason` và metadata rule nếu có.

2.9.3 Tiêu chí đánh giá API và dữ liệu

Đối với API, trọng tâm kiểm thử là contract và tính nhất quán của dữ liệu. Các mutation phải validate input, ghi dữ liệu đúng ràng buộc, tạo audit log trong cùng transaction và làm mất

hiệu lực cache liên quan. Các endpoint đọc phải trả về cấu trúc response ổn định, hỗ trợ phân trang khi cần và không để lộ dữ liệu nội bộ.

Đối với Evaluation API, response luôn phải có dạng evaluation-shaped kể cả khi tài nguyên không tồn tại hoặc xảy ra lỗi. Điều này giúp SDK và Demo Application xử lý fallback bằng cùng một contract thay vì phải phân nhánh theo nhiều kiểu lỗi HTTP khác nhau.

2.9.4 Tiêu chí đánh giá Dashboard và Demo Application

Dashboard được đánh giá theo khả năng hỗ trợ thao tác quản trị an toàn: hiển thị đúng trạng thái lifecycle/runtime, vô hiệu hóa thao tác không đủ quyền, phản ánh lỗi validation và cho phép theo dõi audit log/statistics. Demo Application được đánh giá theo khả năng chứng minh việc thay đổi cấu hình Feature Flag có thể làm thay đổi hành vi ứng dụng mà không cần deploy lại.

Các kịch bản demo được chọn để bao phủ những giá trị quan trọng nhất của sản phẩm: bật/tắt toàn cục, targeting theo user hoặc role, rollout theo phần trăm, flag kill switch, group kill switch, audit log, statistics và fallback của SDK.

2.9.5 Tiêu chí triển khai và trình diễn

Quy trình triển khai được đánh giá bằng khả năng khởi tạo hệ thống trong môi trường sạch. Docker Compose phải khởi chạy được PostgreSQL, Backend API, Admin Dashboard và Demo Application; migration và seed data phải có thể chạy lại theo cách idempotent; các service phải giao tiếp đúng qua cấu hình URL và biến môi trường.

Tiêu chí cuối cùng là khả năng trình bày: sau khi khởi chạy, người xem có thể đi từ Dashboard sang Demo Application, thay đổi cấu hình flag và quan sát kết quả runtime cùng audit/statistics phát sinh.

Chương 3

Kết quả thực hiện và đánh giá

3.1 Môi trường và cấu trúc triển khai

3.1.1 Cấu trúc mã nguồn

Nhằm tách biệt các thành phần của hệ thống nhưng vẫn thuận tiện cho việc phát triển, kiểm thử và triển khai, toàn bộ mã nguồn được tổ chức theo mô hình monorepo. Kiến trúc này cho phép Backend API, Admin Dashboard, Demo Application và JavaScript SDK cùng sử dụng chung quy trình quản lý phiên bản, phụ thuộc và môi trường phát triển. Đồng thời, việc đặt các thành phần trong cùng một kho mã nguồn giúp giảm chi phí đồng bộ API contract, chia sẻ kiểu dữ liệu và triển khai Docker Compose cho toàn bộ hệ thống [12].

Tổ chức monorepo

Dự án được tổ chức theo mô hình workspace với hai thư mục chính là `apps` và `packages`. Thư mục `apps` chứa các ứng dụng có thể triển khai độc lập, bao gồm Backend API, Admin Dashboard và Demo Application. Thư mục `packages` chứa JavaScript SDK dùng chung để các ứng dụng khách gọi Evaluation API.

Cấu trúc này giúp giảm trùng lặp mã nguồn, dễ dàng chia sẻ các kiểu dữ liệu giữa các thành phần và hỗ trợ mở rộng thêm ứng dụng mới mà không ảnh hưởng đến kiến trúc hiện có. Việc phát triển theo monorepo cũng phù hợp với quy trình tích hợp và triển khai liên tục khi toàn bộ hệ thống được quản lý trong cùng một repository [4, 12].

Cấu trúc Backend API

Backend API được xây dựng bằng NestJS theo kiến trúc module nhằm tách biệt các miền chức năng của hệ thống. Mỗi module đảm nhiệm một nghiệp vụ riêng như quản lý project, feature flag, flag group, rule evaluation, audit log, authentication, sample user và thống kê evaluation.

Tầng controller chịu trách nhiệm tiếp nhận và xác thực request, tầng service xử lý nghiệp vụ, còn tầng repository thực hiện truy cập cơ sở dữ liệu thông qua Prisma ORM. Các DTO được sử dụng để kiểm tra dữ liệu đầu vào, trong khi middleware, guard, interceptor và exception filter đảm nhiệm các chức năng cắt ngang như xác thực, phân quyền, ghi nhận request ID, logging và xử lý lỗi thống nhất. Kiến trúc phân lớp này giúp tăng khả năng bảo trì, kiểm thử và mở rộng hệ thống khi bổ sung các loại rule hoặc endpoint mới.

Cấu trúc Admin Dashboard

Admin Dashboard được xây dựng bằng React theo hướng component-based. Giao diện được chia thành các nhóm thành phần độc lập như trang quản lý project, feature flag, flag group, audit log, statistics và màn hình cấu hình rule.

Các thành phần dùng chung như layout, biểu mẫu, bảng dữ liệu, hộp thoại xác nhận, badge trạng thái và hook xử lý API được tách riêng để tái sử dụng trong toàn bộ ứng dụng. Dữ liệu được lấy từ Backend API thông qua các service riêng biệt, giúp tách logic giao diện khỏi logic giao tiếp mạng và đơn giản hóa việc bảo trì khi API thay đổi.

Cấu trúc Demo Application và SDK

Demo Application là một ứng dụng React độc lập dùng để minh họa việc tích hợp Feature Flag vào ứng dụng khách. Thay vì gọi trực tiếp Evaluation API, ứng dụng sử dụng JavaScript SDK được phát triển trong thư mục `packages/js-sdk`. SDK chịu trách nhiệm xây dựng request, gửi Evaluation API, kiểm tra tính hợp lệ của response và trả về kết quả dưới dạng giao diện lập trình thống nhất cho ứng dụng.

Việc tách SDK thành một package riêng giúp giảm sự phụ thuộc giữa ứng dụng khách và Backend API, đồng thời tạo điều kiện tái sử dụng cho nhiều ứng dụng khác nhau. Cách tổ chức này cũng phù hợp với mô hình Feature Flag Platform trong thực tế, nơi client chỉ cần tích hợp SDK mà không cần quan tâm đến chi tiết triển khai của Evaluation Engine [3, 13].

3.1.2 Môi trường phát triển

Để bảo đảm tính ổn định, khả năng tái lập môi trường và thuận tiện trong quá trình phát triển, hệ thống được xây dựng trên nền tảng Node.js với kiến trúc monorepo. Backend API, Admin Dashboard, Demo Application và JavaScript SDK cùng sử dụng chung hệ sinh thái công cụ, đồng thời cơ sở dữ liệu được quản lý thông qua Prisma ORM và PostgreSQL. Ngoài ra, dự án tích hợp các công cụ kiểm thử, kiểm tra chất lượng mã nguồn và tài liệu hóa API nhằm nâng cao khả năng bảo trì và mở rộng hệ thống.

Runtime và package manager

Toàn bộ hệ thống được phát triển bằng TypeScript trên nền tảng Node.js. Backend sử dụng NestJS, trong khi Admin Dashboard và Demo Application được xây dựng bằng React. Các thành phần trong monorepo được quản lý bằng npm workspaces, cho phép chia sẻ package nội bộ, đồng bộ phiên bản phụ thuộc và đơn giản hóa quá trình cài đặt.

Việc sử dụng cùng một runtime giúp giảm sự khác biệt giữa các thành phần của hệ thống, đồng thời tạo điều kiện thuận lợi cho việc xây dựng quy trình build, kiểm thử và triển khai thống nhất. Đây cũng là mô hình thường được áp dụng trong các hệ thống web hiện đại theo quy trình DevOps [4].

Database và công cụ ORM

Hệ thống sử dụng PostgreSQL làm hệ quản trị cơ sở dữ liệu chính. PostgreSQL đáp ứng tốt các yêu cầu về tính toàn vẹn dữ liệu, hỗ trợ transaction, khóa ngoại, chỉ mục và kiểu dữ liệu JSONB phục vụ lưu trữ linh hoạt các tham số rule, thuộc tính người dùng và snapshot của audit log [11].

Lớp truy cập dữ liệu được xây dựng bằng Prisma ORM. Prisma cung cấp cơ chế mô hình hóa dữ liệu thông qua schema, hỗ trợ migration, sinh mã truy cập cơ sở dữ liệu theo kiểu dữ liệu tĩnh và giảm đáng kể lượng mã SQL thủ công. Việc sử dụng Prisma cũng giúp tăng tính an toàn kiểu dữ liệu, đơn giản hóa thao tác CRUD và thuận tiện khi mở rộng mô hình dữ liệu trong tương lai.

Công cụ kiểm thử và kiểm tra mã nguồn

Dự án sử dụng Jest để xây dựng các bài kiểm thử đơn vị cho các thành phần nghiệp vụ quan trọng, đặc biệt là Rule Evaluation Engine, cache, SDK và các hàm tiện ích. Các trường hợp kiểm thử bao gồm đánh giá theo user allowlist, role, percentage rollout, group/flag kill switch, default off, `NOT_FOUND`, `INVALID_CONTEXT` và các tình huống ngoại lệ nhằm bảo đảm tính đúng đắn của thuật toán evaluation.

Bên cạnh đó, ESLint được sử dụng để kiểm tra chất lượng mã nguồn và phát hiện các lỗi phổ biến trong quá trình phát triển. Việc kết hợp TypeScript với ESLint giúp chuẩn hóa phong cách lập trình, hạn chế lỗi tiềm ẩn và nâng cao khả năng bảo trì của toàn bộ hệ thống.

Công cụ tài liệu hóa API

Backend API tích hợp OpenAPI thông qua Swagger để tự động sinh tài liệu REST API. Các endpoint được mô tả đầy đủ về request, response, tham số đầu vào, mã lỗi và kiểu dữ liệu, giúp lập trình viên dễ dàng kiểm thử và tích hợp với các ứng dụng khách.

Việc sử dụng Swagger giúp tài liệu API luôn đồng bộ với mã nguồn, giảm công sức cập nhật tài liệu thủ công và hỗ trợ kiểm thử trực tiếp ngay trên giao diện trình duyệt. Cách tiếp cận này phù hợp với các hệ thống Feature Flag hiện đại, nơi Evaluation API đóng vai trò là điểm tích hợp giữa nền tảng quản trị và các ứng dụng khách [13].

3.1.3 Database migration và seed data

Để bảo đảm cơ sở dữ liệu luôn đồng bộ với mã nguồn và có thể tái tạo trên nhiều môi trường khác nhau, hệ thống sử dụng Prisma Migrate để quản lý schema và lịch sử thay đổi cơ sở dữ liệu. Bên cạnh đó, các tập dữ liệu mẫu được chuẩn bị sẵn nhằm hỗ trợ phát triển, kiểm thử và trình diễn các kịch bản Feature Flag mà không cần nhập dữ liệu thủ công.

Khởi tạo database schema

Toàn bộ mô hình dữ liệu của hệ thống được định nghĩa trong Prisma Schema và sinh tự động thành các bảng trong PostgreSQL thông qua Prisma Migrate. Schema bao gồm các thực thể chính như Project, Environment, Feature Flag, Flag Group, FlagEnvironmentConfig, FlagRule, SampleUserContext, AuditLogEntry và các bảng phục vụ thống kê evaluation.

Các ràng buộc toàn vẹn như khóa chính, khóa ngoại, chỉ mục, giá trị duy nhất và các quan hệ

một-nhiều được khai báo trực tiếp trong schema. Ngoài các kiểu dữ liệu cơ bản, PostgreSQL còn sử dụng kiểu dữ liệu JSONB để lưu trữ linh hoạt các tham số rule, thuộc tính người dùng và snapshot của audit log, giúp hệ thống dễ dàng mở rộng khi bổ sung các loại rule mới [11].

Quản lý migration

Mỗi thay đổi đối với mô hình dữ liệu đều được quản lý dưới dạng migration độc lập. Prisma tự động sinh các tập lệnh SQL tương ứng và lưu lại lịch sử migration trong cơ sở dữ liệu, bảo đảm mọi môi trường phát triển và triển khai đều sử dụng cùng một phiên bản schema.

Cách tiếp cận này giúp kiểm soát sự thay đổi của cơ sở dữ liệu theo thời gian, hỗ trợ cập nhật hoặc khôi phục schema khi cần thiết, đồng thời giảm rủi ro sai khác giữa môi trường phát triển, kiểm thử và production.

Seed project, environment và Feature Flag

Sau khi hoàn tất migration, hệ thống cung cấp các script seed để tạo dữ liệu mẫu phục vụ quá trình phát triển và trình diễn. Dữ liệu khởi tạo bao gồm một hoặc nhiều project, các environment mặc định như `development`, `staging` và `production`, các flag group cùng nhiều feature flag đại diện cho các kịch bản phổ biến.

Mỗi feature flag được cấu hình với trạng thái, serving mode, kill switch và các rule phù hợp để minh họa các chiến lược rollout như bật hoặc tắt toàn cục, targeting theo user, targeting theo role, percentage rollout và group kill switch. Nhờ đó, các chức năng của Evaluation Engine và Dashboard có thể được kiểm thử ngay sau khi khởi tạo dữ liệu.

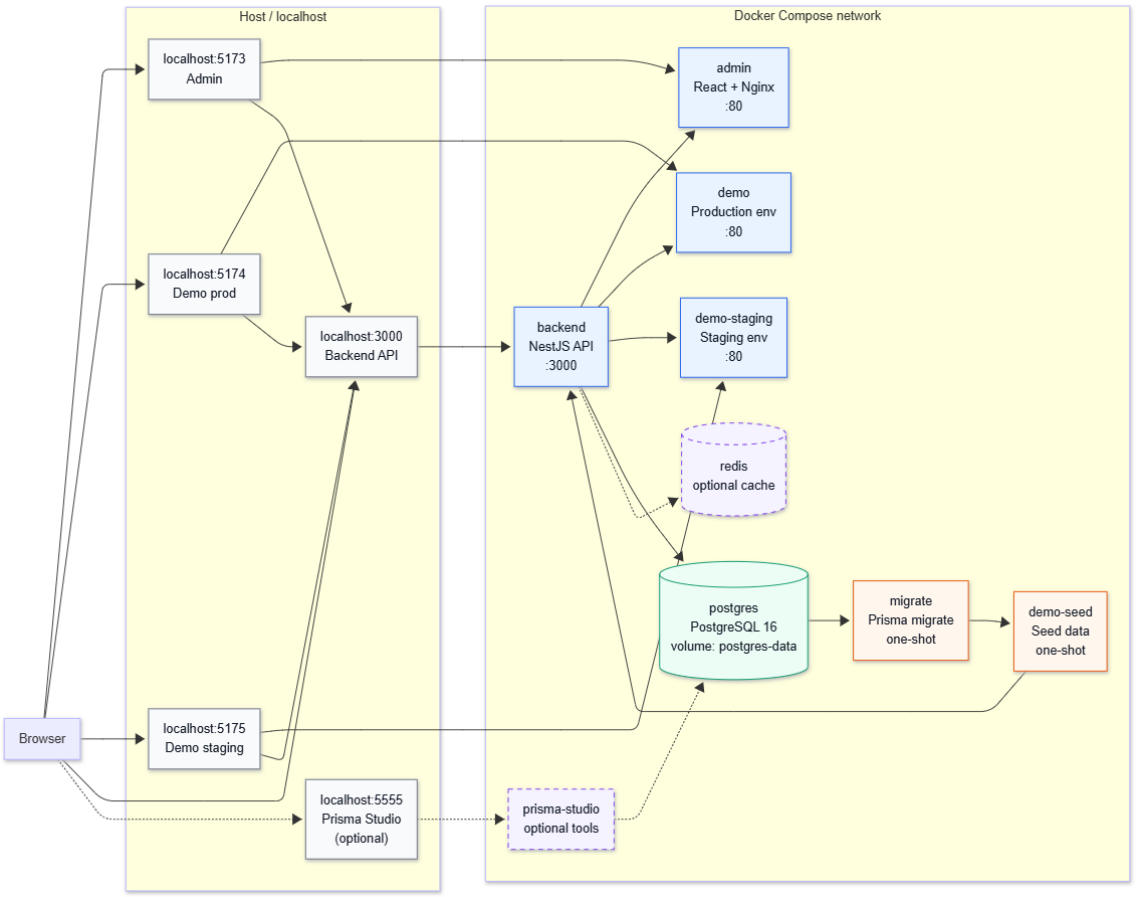
Seed sample user và thuộc tính organization mẫu

Hệ thống đồng thời tạo tập sample user và thuộc tính organization mẫu nhằm phục vụ các kịch bản đánh giá Feature Flag. Mỗi người dùng được gán một `targetingKey` duy nhất cùng với các thuộc tính như vai trò, organization và các metadata cần thiết cho quá trình evaluation.

Việc chuẩn bị sẵn dữ liệu mẫu giúp dễ dàng kiểm thử các trường hợp targeting theo user ID, role và rollout theo organization-level targetingKey trong demo, đồng thời bảo đảm các kết quả percentage rollout luôn ổn định nhờ sử dụng cùng một định danh người dùng trong quá trình đánh giá. Các tập dữ liệu này cũng được sử dụng để minh họa các kịch bản demo và hỗ trợ kiểm thử tự động của hệ thống.

3.1.4 Triển khai bằng Docker Compose

Để đơn giản hóa quá trình cài đặt và bảo đảm tính nhất quán giữa các môi trường phát triển, hệ thống được triển khai bằng Docker Compose. Cách tiếp cận này cho phép khởi tạo đồng thời các thành phần của nền tảng Feature Flag thông qua một cấu hình thống nhất, giảm sự phụ thuộc vào môi trường cài đặt của từng máy và hỗ trợ tái tạo hệ thống một cách nhanh chóng [12].



Hình 3.1: Các service chính trong quy trình triển khai bằng Docker Compose

Các service trong Docker Compose

Docker Compose được sử dụng để quản lý các service chính của hệ thống. Trong phiên bản hiện tại, các service bao gồm cơ sở dữ liệu PostgreSQL và các ứng dụng chạy trong môi trường Node.js như Backend API, Admin Dashboard và Demo Application. Khi cần mở rộng, các service này có thể được bổ sung thêm Redis để cache snapshot cấu hình evaluation hoặc các thành phần giám sát mà không làm thay đổi kiến trúc tổng thể.

Mỗi service được khai báo độc lập với image, cổng truy cập, biến môi trường, volume và các

phụ thuộc cần thiết. Việc quản lý tập trung giúp quá trình khởi động, dừng và cập nhật toàn bộ hệ thống chỉ cần thực hiện bằng một số lệnh Docker Compose thống nhất.

Network giữa các service

Các service được kết nối thông qua mạng nội bộ do Docker Compose tự động tạo. Nhờ đó, Backend API có thể truy cập PostgreSQL thông qua tên service thay vì địa chỉ IP cố định, trong khi Admin Dashboard và Demo Application giao tiếp với Backend API thông qua các endpoint được cấu hình trong biến môi trường.

Việc sử dụng mạng nội bộ giúp cô lập các thành phần của hệ thống với môi trường bên ngoài, giảm sự phụ thuộc vào cấu hình mạng của máy chủ và đơn giản hóa quá trình triển khai trên nhiều thiết bị khác nhau [12].

Volume lưu trữ dữ liệu

Để bảo đảm dữ liệu không bị mất khi container được khởi động lại hoặc cập nhật phiên bản, PostgreSQL sử dụng Docker Volume để lưu trữ dữ liệu lâu dài. Toàn bộ schema, dữ liệu nghiệp vụ, audit log và các thông tin cấu hình Feature Flag được lưu trên volume thay vì bên trong container.

Việc tách dữ liệu khỏi vòng đời của container giúp hệ thống dễ dàng sao lưu, phục hồi và nâng cấp mà không ảnh hưởng đến dữ liệu đã tồn tại. Đây cũng là phương pháp triển khai được khuyến nghị đối với các dịch vụ sử dụng cơ sở dữ liệu [12].

Biến môi trường và cấu hình runtime

Các thông số cấu hình của hệ thống được quản lý thông qua biến môi trường nhằm tách biệt mã nguồn với thông tin triển khai. Các biến quan trọng bao gồm chuỗi kết nối PostgreSQL, cổng dịch vụ, khóa bí mật phục vụ xác thực, cấu hình CORS, địa chỉ Backend API và các tham số runtime khác.

Việc sử dụng biến môi trường giúp cùng một mã nguồn có thể triển khai trên nhiều môi trường như development, staging và production mà không cần chỉnh sửa chương trình. Đồng thời, các thông tin nhạy cảm không được ghi trực tiếp trong mã nguồn, góp phần nâng cao tính bảo mật và khả năng quản lý cấu hình của hệ thống [9, 12].

3.1.5 Quy trình khởi chạy hệ thống

Hệ thống được thiết kế để có thể khởi tạo nhanh trên một môi trường mới thông qua quy trình gồm chuẩn bị môi trường, khởi tạo cơ sở dữ liệu, sinh dữ liệu mẫu và khởi chạy các dịch vụ. Các bước này bảo đảm toàn bộ thành phần của nền tảng Feature Flag được đồng bộ trước khi đưa vào sử dụng hoặc trình diễn.

Khởi tạo môi trường

Bước đầu tiên là cài đặt các công cụ cần thiết như Node.js, npm và Docker Desktop. Sau khi tải mã nguồn, toàn bộ các package trong monorepo được cài đặt thông qua npm workspaces để đồng bộ các phụ thuộc giữa Backend API, Admin Dashboard, Demo Application và JavaScript SDK.

Tiếp theo, các biến môi trường được cấu hình thông qua tệp cấu hình tương ứng. Các tham số như chuỗi kết nối PostgreSQL, cổng dịch vụ, khóa bí mật và địa chỉ Backend API được thiết lập trước khi khởi động hệ thống nhằm bảo đảm các thành phần có thể giao tiếp với nhau một cách chính xác.

Chạy migration và seed

Sau khi cơ sở dữ liệu PostgreSQL được khởi động, Prisma Migrate được sử dụng để tạo hoặc cập nhật schema theo phiên bản mới nhất của hệ thống. Quá trình migration đồng thời thiết lập các bảng dữ liệu, khóa ngoại, chỉ mục và các ràng buộc toàn vẹn cần thiết.

Khi migration hoàn tất, các script seed được thực thi để tạo dữ liệu mẫu phục vụ phát triển và kiểm thử. Dữ liệu khởi tạo bao gồm project, environment, feature flag, flag group, rule và sample user, cho phép hệ thống có thể sử dụng ngay sau khi cài đặt mà không cần cấu hình thủ công.

Khởi chạy các service

Các service của hệ thống được khởi động thông qua Docker Compose hoặc chạy trực tiếp trong môi trường phát triển. PostgreSQL được khởi động trước, tiếp theo là Backend API, Admin Dashboard và Demo Application. Trong quá trình khởi động, Backend API tự động kết nối đến cơ sở dữ liệu và sẵn sàng phục vụ các yêu cầu từ Dashboard cũng như Demo Application.

Việc khởi chạy theo thứ tự giúp bảo đảm các dịch vụ phụ thuộc luôn sẵn sàng trước khi ứng

dụng bắt đầu xử lý yêu cầu, đồng thời giảm khả năng phát sinh lỗi kết nối trong giai đoạn khởi động [12].

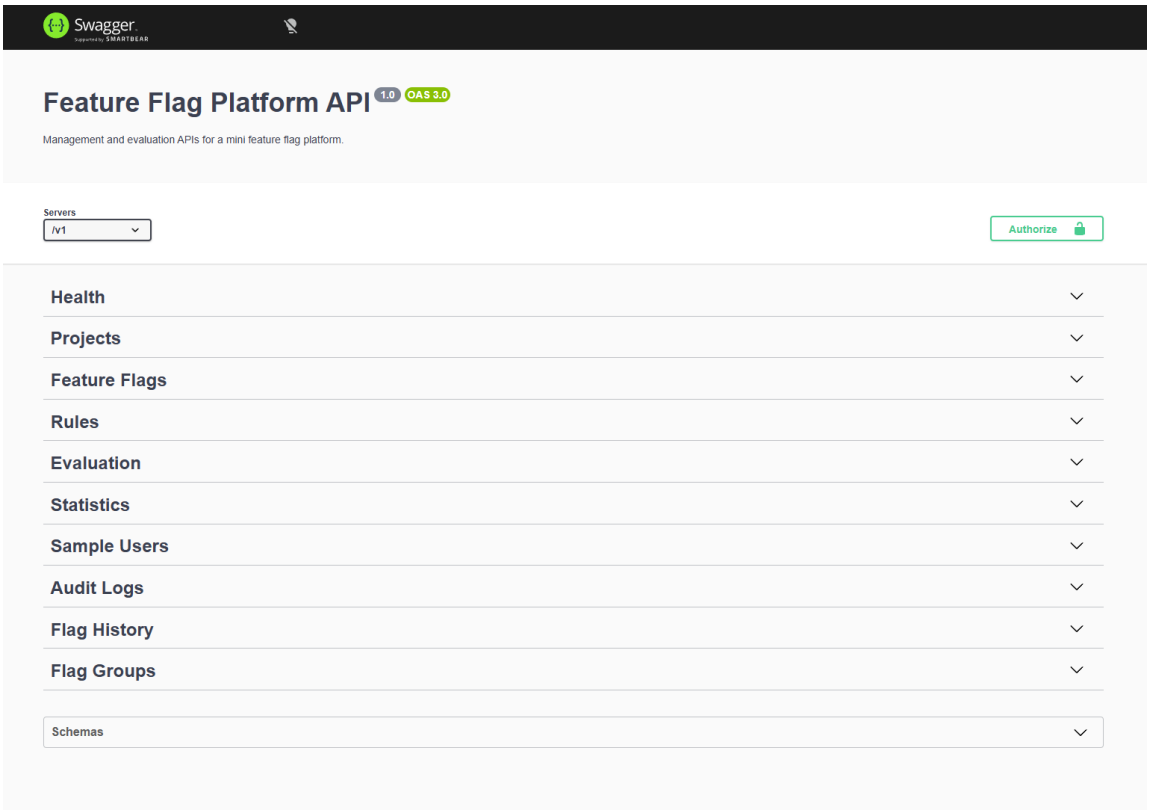
Kiểm tra trạng thái hệ thống

Sau khi toàn bộ service được khởi động, trạng thái hệ thống được kiểm tra bằng cách truy cập giao diện Admin Dashboard, Demo Application và tài liệu Swagger của Backend API. Các endpoint quản lý project, feature flag và evaluation được sử dụng để xác nhận Backend API đã kết nối thành công với cơ sở dữ liệu.

Cuối cùng, một số kịch bản đánh giá Feature Flag như bật hoặc tắt toàn cục, targeting theo role và percentage rollout được thực hiện trên Demo Application nhằm xác nhận Rule Evaluation Engine, Dashboard và cơ sở dữ liệu hoạt động đúng theo thiết kế trước khi tiến hành kiểm thử hoặc trình diễn hệ thống.

3.2 Kết quả xây dựng Backend API

Backend API đã được triển khai bằng NestJS và Prisma, giữ vai trò trung tâm cho cả management plane và Evaluation API. Phần này chỉ tóm tắt kết quả hiện thực so với thiết kế; các nguyên tắc chi tiết đã được trình bày ở Chương 2.



Hình 3.2: Minh chứng giao diện tài liệu Backend API/OpenAPI đã được sinh từ source code

Bảng 3.1: Tổng hợp các nhóm endpoint đã triển khai trong Backend API

Nhóm API	Endpoint chính	Vai trò trong hệ thống
Health	`GET /v1/health`	Kiểm tra trạng thái backend và health check cho Docker Compose.
Projects	`GET/POST /v1/projects`; `GET/PATCH/DELETE /v1/projects/{projectKey}`	Quản lý project, soft delete và phạm vi cấu hình cấp cao nhất.
Feature Flags	`GET/POST /v1/projects/{projectKey}/flags`; `PATCH/DELETE /flags/{flagKey}`; archive/restore	Quản lý định nghĩa flag, lifecycle và cấu hình runtime mặc định.
Rules	`GET/PUT /v1/projects/{projectKey}/flags/{flagKey}/rules`	Đọc hoặc thay thế toàn bộ rule set của một flag.
Flag Groups	`GET/POST/PATCH/DELETE /groups`; `PUT /groups/{groupKey}/config`; `PUT/DELETE /flags/{flagKey}/group`	Quản lý nhóm flag, group kill switch và gán/gỡ flag khỏi group.
Evaluation	`POST /v1/evaluate`	Data-plane endpoint công khai cho SDK/Demo App, trả kết quả fail-closed có reason code.
Audit/History	`GET /audit-logs`; `GET /flags/{flagKey}/history`	Truy vấn audit log append-only và lịch sử cấu hình theo flag.
Statistics	`GET /stats/flags`; `GET /flags/{flagKey}/stats`	Trả dữ liệu aggregate số lần evaluation theo flag, reason, kết quả và thời gian.
Sample Users	`GET/POST /sample-users`; `DELETE /sample-users/{targetingKey}`	Quản lý context mẫu dùng cho Dashboard và Demo Application.

3.2.1 Quản lý project và environment nội bộ

Backend hỗ trợ quản lý project và sử dụng environment làm phạm vi cấu hình/evaluation. Environment xuất hiện trong schema, seed data, repository và Evaluation API; Dashboard hiện chưa mở màn hình CRUD environment riêng.

Kết quả triển khai API project

API project cho phép tạo, cập nhật, liệt kê và truy vấn project theo khóa định danh ổn định. Các ràng buộc dữ liệu được kiểm tra ở DTO và database.

Kết quả triển khai environment trong repository và evaluation

Repository truy xuất cấu hình theo project key và environment key, giúp cùng một flag có thể có trạng thái khác nhau giữa staging và production.

Environment mặc định

Seed data tạo sẵn environment mặc định để demo có thể chạy ngay sau migration/seed mà không cần cấu hình thủ công.

Validation và ràng buộc dữ liệu

Backend chuẩn hóa lỗi validation và giữ các ràng buộc duy nhất ở tầng database để tránh trạng thái trùng khóa.

3.2.2 Quản lý Feature Flag

API Feature Flag đã đáp ứng các thao tác chính: tạo, truy vấn, cập nhật thông tin, archive/restore và cập nhật cấu hình theo environment.

Tạo và truy vấn Feature Flag

Khi tạo flag, backend kiểm tra project tồn tại và khóa flag chưa bị trùng trong project. API truy vấn trả về dữ liệu đủ cho Dashboard hiển thị danh sách và chi tiết.

Cập nhật thông tin flag

Các trường mô tả được cập nhật qua endpoint quản trị và được ghi nhận audit log.

Archive và restore flag

Archive/restore được triển khai như thay đổi trạng thái vòng đời thay vì xóa vật lý, giúp giữ lịch sử cấu hình.

Quản lý cấu hình theo environment

Cấu hình runtime như enabled, serving mode và kill switch được lưu theo environment, hỗ trợ demo staging/production.

3.2.3 Quản lý rule và flag group

Backend cung cấp endpoint thay thế rule set, đổi priority và quản lý group kill switch. Cách triển khai tập trung rule mutation giúp audit và cache invalidation nhất quán.

Rule user allowlist

Rule allowlist lưu danh sách user ID trong parameters và được validation trước khi lưu.

Rule role targeting

Rule role targeting lưu danh sách role hợp lệ, phục vụ kịch bản demo theo nhóm người dùng.

Rule percentage rollout

Rule percentage rollout lưu tỷ lệ phần trăm trong khoảng hợp lệ và giao phần quyết định bucket cho engine.

Quản lý độ ưu tiên rule

Backend lưu priority cho rule cùng loại; engine vẫn áp dụng type precedence cố định để kết quả dễ dự đoán.

Group kill switch

Flag group và group config đã được triển khai để vô hiệu hóa đồng thời nhiều flag trong một environment.

3.2.4 Evaluation API

Evaluation API trả response thống nhất cho ứng dụng khách, bao gồm `enabled`, `reason`, `projectKey`, `flagKey` và các metadata cần thiết.

Contract request/response

Request nhận project key, environment key, flag key và context. Response không phơi bày cấu hình quản trị nhạy cảm mà chỉ trả quyết định đã được chuẩn hóa.

Snapshot và cache

Backend sử dụng snapshot cấu hình để giảm truy vấn lặp. Cache miss hoặc invalidation đều quay lại repository để lấy dữ liệu mới.

Kết quả fail-closed

Các trường hợp không tìm thấy project/flag hoặc lỗi nội bộ đều trả về kết quả tắt an toàn kèm reason tương ứng.

3.2.5 Audit log và lịch sử cấu hình

Audit log đã được tích hợp vào các mutation quản trị quan trọng và được dùng làm cơ sở cho lịch sử cấu hình.

Ghi audit log cho mutation

Các thao tác tạo/cập nhật/archive/restore flag, thay đổi rule và group kill switch đều tạo audit entry sau khi kiểm tra quyền.

Lưu snapshot trước và sau

Audit entry lưu before/after snapshot để người dùng có thể so sánh thay đổi mà không cần suy luận từ nhiều bản ghi riêng lẻ.

Truy vấn và lọc audit log

API hỗ trợ truy vấn danh sách audit log và lọc theo các trường phục vụ Dashboard.

Bảo vệ audit log khỏi sửa và xóa

Repository không cung cấp luồng sửa/xóa audit log trong ứng dụng, giữ đặc tính append-only ở tầng nghiệp vụ.

Theo dõi lịch sử cấu hình

History API tận dụng audit log để hiển thị chuỗi thay đổi theo đối tượng cấu hình.

3.2.6 Validation và xử lý lỗi

Backend chuẩn hóa validation, lỗi nghiệp vụ và lỗi ngoài dự kiến để client nhận được phản hồi nhất quán.

Validation DTO

DTO kiểm tra định dạng key, kiểu dữ liệu và giới hạn giá trị trước khi request đi vào service.

Chuẩn hóa phản hồi lỗi

Lỗi được trả về theo cấu trúc thống nhất, giúp Dashboard hiển thị thông báo rõ ràng.

Request ID

Request ID được dùng để liên kết phản hồi lỗi, audit log và quá trình debug.

Xử lý lỗi toàn cục

Exception filter toàn cục tránh rò rỉ stack trace và chuyển lỗi ngoài dự kiến thành phản hồi an toàn.

3.2.7 Phân quyền người dùng quản trị

RBAC demo đã được triển khai ở backend để bảo vệ endpoint mutation và cung cấp trạng thái quyền cho Dashboard.

Permission matrix tập trung

Permission matrix định nghĩa quyền theo role ở một nơi, tránh lặp logic phân quyền trong nhiều controller.

Bảo vệ mutation endpoint

Guard kiểm tra actor và quyền trước khi cho phép ghi dữ liệu.

Phản ánh quyền trên Dashboard

Backend trả thông tin role/permission để Dashboard hiển thị trạng thái phù hợp.

Kết quả xử lý truy cập không hợp lệ

Request thiếu quyền bị từ chối bằng lỗi chuẩn hóa và không tạo mutation/audit sai lệch.

3.2.8 Cache và evaluation metrics

Backend đã bổ sung snapshot cache và thống kê aggregate cho Evaluation API.

Cơ chế caching

Cache lưu snapshot theo project/environment/flag, không lưu dữ liệu context người dùng.

Cache invalidation

Các mutation ảnh hưởng đến evaluation đều gọi invalidation để snapshot cũ không tiếp tục được dùng.

Ghi nhận số lần evaluation

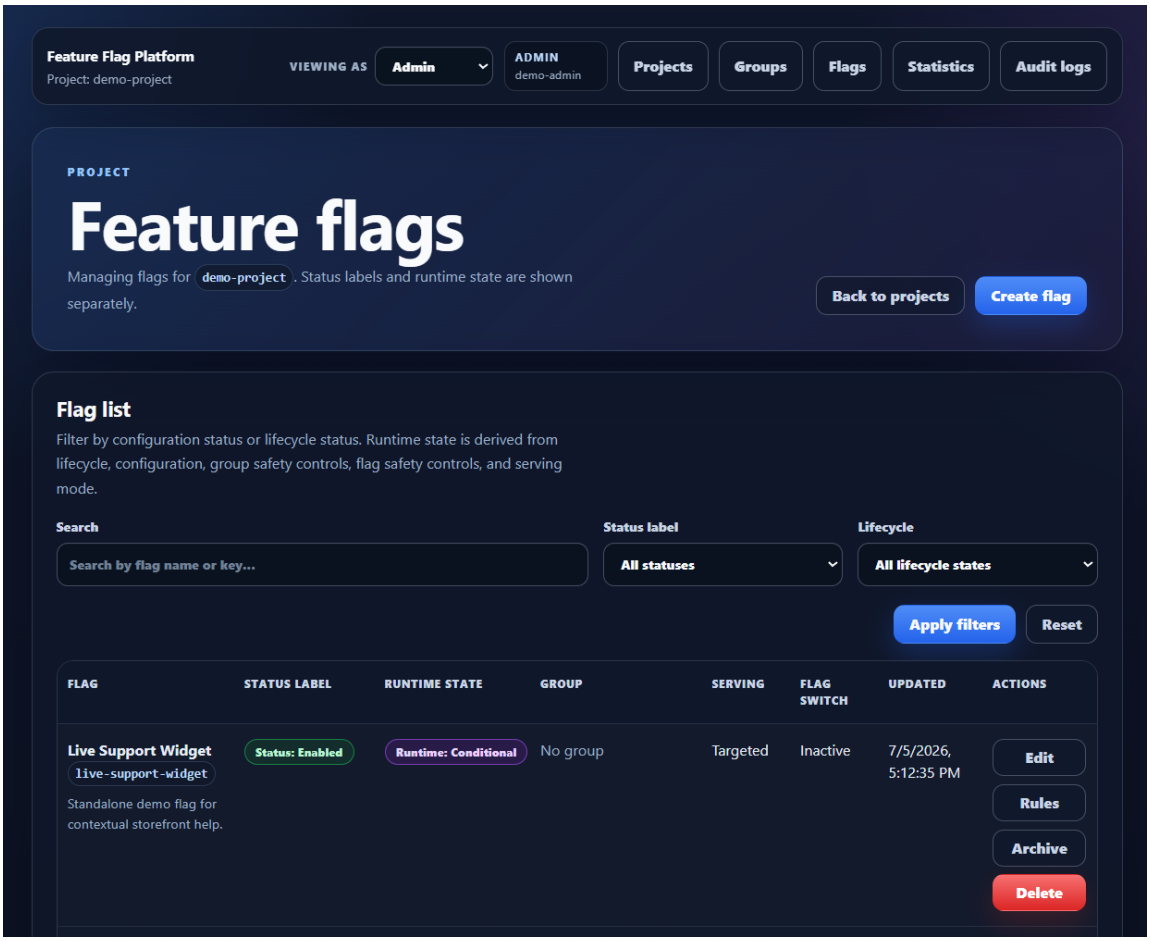
Sau mỗi evaluation, backend ghi nhận thống kê aggregate theo flag, environment, reason và kết quả bật/tắt.

Truy vấn dữ liệu thống kê

Dashboard sử dụng API statistics để hiển thị tổng số lượt evaluation, phân bố theo flag và xu hướng theo thời gian.

3.3 Kết quả xây dựng Frontend Dashboard

Admin Dashboard đã được xây dựng để kiểm chứng các luồng quản trị chính của nền tảng. Mục này tập trung vào kết quả giao diện và trạng thái hiển thị, không lặp lại thiết kế API hay rule engine.



Hình 3.3: Minh chứng giao diện Admin Dashboard đã được xây dựng

3.3.1 Màn hình quản lý project

Màn hình project là điểm vào của Dashboard, cho phép người dùng xem danh sách project và truy cập khu vực quản trị Feature Flag của từng project.

Danh sách project

Danh sách hiển thị khóa, tên, mô tả và trạng thái cơ bản của project để người dùng nhanh chóng chọn phạm vi làm việc.

Tạo và chỉnh sửa project

Biểu mẫu tạo/sửa project gọi Management API và hiển thị lỗi validation từ backend khi dữ liệu không hợp lệ.

Truy cập chi tiết project

Từ danh sách, người dùng có thể mở chi tiết project để quản lý flag, group, audit log và statistics.

Phạm vi environment trong Dashboard hiện tại

Dashboard hiện sử dụng environment đã được seed và chưa cung cấp màn hình CRUD environment riêng.

3.3.2 Màn hình quản lý Feature Flag

Màn hình Feature Flag hiển thị danh sách flag trong project và cung cấp các thao tác quản trị phổ biến.

Danh sách Feature Flag

Bảng flag hiển thị flag key, tên, mô tả, trạng thái lifecycle và thông tin runtime tóm tắt.

Tìm kiếm và lọc flag

Người dùng có thể lọc danh sách để tìm flag cần thao tác trong các project có nhiều cấu hình.

Hiển thị trạng thái runtime

Dashboard phân biệt trạng thái lifecycle với trạng thái runtime để tránh nhầm lẫn giữa flag đã archive và flag đang tắt do cấu hình.

Archive và restore flag

Các thao tác archive/restore được đặt ở vị trí rõ ràng và có phản hồi sau khi backend cập nhật thành công.

3.3.3 Màn hình tạo và chỉnh sửa flag

Biểu mẫu flag tập trung vào các trường cần thiết để người dùng cấu hình nhanh nhưng vẫn giữ dữ liệu hợp lệ.

Nhập thông tin định danh

Flag key được nhập theo định dạng ổn định, dùng làm định danh cho Evaluation API và ứng dụng khách.

Chọn trạng thái và serving mode

Người dùng chọn trạng thái cấu hình theo environment và serving mode phù hợp với kịch bản phát hành.

Validation biểu mẫu

Frontend kiểm tra các lỗi đơn giản trước khi gửi request, còn backend vẫn là nguồn validation cuối cùng.

Hiển thị lỗi từ Backend API

Khi backend trả lỗi chuẩn hóa, Dashboard hiển thị thông báo gần trường nhập hoặc ở vùng cảnh báo chung.

3.3.4 Màn hình cấu hình rule

Màn hình rule cho phép người dùng cấu hình ba loại rule đã hỗ trợ và quan sát thứ tự áp dụng mà không cần hiểu chi tiết mã nguồn engine.

Cấu hình user allowlist

Giao diện nhận danh sách user ID mẫu và gửi về backend dưới dạng parameters của rule allowlist.

Cấu hình role targeting

Giao diện cho phép chọn một hoặc nhiều role mẫu để phục vụ demo phát hành theo nhóm người dùng.

Cấu hình percentage rollout

Người dùng nhập hoặc điều chỉnh tỷ lệ rollout trong khoảng hợp lệ; frontend chặn các giá trị ngoài miền trước khi gửi.

Thay đổi độ ưu tiên rule

Dashboard hỗ trợ sắp xếp rule cùng loại bằng priority, sau đó lưu lại qua API quản trị.

Hiển thị tác động của rule

Mỗi rule được trình bày với loại, tham số và trạng thái để người quản trị kiểm tra trước khi lưu cấu hình.

3.3.5 Màn hình quản lý flag group

Màn hình group chứng minh khả năng quản trị nhiều flag liên quan trong cùng một thao tác.

Danh sách flag group

Danh sách group hiển thị group key, tên, số lượng flag thành viên và trạng thái kill switch theo environment.

Quản lý thành viên của group

Dashboard hỗ trợ thêm hoặc loại bỏ flag khỏi group thông qua danh sách chọn trực quan.

Kích hoạt group kill switch

Công tắc group kill switch gửi mutation đến backend và làm mới danh sách sau khi thành công.

Xác nhận thao tác nguy hiểm

Các thao tác ảnh hưởng nhiều flag yêu cầu xác nhận để giảm rủi ro vận hành nhầm.

3.3.6 Màn hình audit log

Màn hình audit log giúp người dùng kiểm tra lịch sử thay đổi phát sinh từ các mutation quản trị.

Danh sách thay đổi

Bảng audit log sắp xếp theo thời gian và hiển thị actor, action, target, request ID cùng thời điểm thao tác.

Lọc theo actor và action

Bộ lọc giúp thu hẹp lịch sử khi cần điều tra một người dùng hoặc một loại thao tác cụ thể.

Hiển thị before và after snapshot

Dashboard trình bày snapshot trước/sau để người dùng nhìn thấy chính xác dữ liệu đã đổi.

Truy vết theo request ID

Request ID được hiển thị để liên kết audit log với phản hồi API hoặc log backend khi debug.

3.3.7 Màn hình thống kê evaluation

Màn hình statistics tổng hợp các lượt evaluation đã được backend ghi nhận.

Tổng số lần evaluation

Dashboard hiển thị tổng số request evaluation trong phạm vi thời gian được chọn.

Thống kê theo Feature Flag

Bảng thống kê theo flag giúp nhận biết flag nào được gọi nhiều trong demo.

Thống kê theo kết quả bật và tắt

Dashboard phân tách số lần enabled/disabled để người dùng đối chiếu với chiến lược rollout.

Biểu đồ theo thời gian

Biểu đồ thời gian minh họa xu hướng evaluation và hỗ trợ trình diễn tác động khi thay đổi cấu hình.

3.3.8 Hiển thị quyền và trạng thái

Dashboard phản ánh role hiện tại và trạng thái quyền để người dùng hiểu thao tác nào khả dụng.

Ẩn hoặc vô hiệu hóa thao tác

Các nút không đủ quyền được ẩn hoặc disabled, trong khi backend vẫn kiểm tra quyền lần cuối.

Hiển thị role hiện tại

Role admin/developer/viewer được hiển thị ở khu vực tài khoản để tránh nhầm lẫn trong demo.

Hiển thị trạng thái lifecycle

Lifecycle status cho biết flag đang active hay archived.

Hiển thị trạng thái runtime

Runtime status cho biết flag đang được phục vụ theo cấu hình nào tại environment hiện hành.

3.4 Kết quả xây dựng Demo Application và SDK

Demo Application và JavaScript SDK được triển khai để chứng minh cách một ứng dụng web tiêu thụ Evaluation API. Phần này mô tả kết quả tích hợp thay vì lặp lại thiết kế engine.

3.4.1 Cấu trúc Demo Application

Demo Application mô phỏng một trang thương mại điện tử nhỏ với các khu vực giao diện chịu ảnh hưởng bởi Feature Flag.

Mô hình ứng dụng thương mại điện tử mẫu

Ứng dụng có các thành phần như danh sách sản phẩm, khuyến mãi, checkout hoặc banner thử nghiệm để minh họa tác động của flag.

Dữ liệu người dùng và organization

Demo cung cấp một số người dùng mẫu với role và organization khác nhau để kiểm tra các kịch bản targeting.

Các tính năng được kiểm soát bằng flag

Mỗi tính năng demo tương ứng với một flag đã seed, giúp người xem thay đổi cấu hình ở Dashboard và quan sát kết quả ở ứng dụng.

Môi trường staging và production

Ứng dụng có thể gửi environment key khác nhau để minh họa cùng một flag nhưng cấu hình khác giữa môi trường.

3.4.2 JavaScript SDK

SDK là lớp client nhỏ, typed và fail-closed, dùng lại contract của Evaluation API.

Khởi tạo SDK client

Client nhận base URL, project/environment mặc định và timeout để chuẩn bị gọi evaluation.

Phương thức evaluate

Phương thức `evaluate` nhận flag key và context, sau đó trả về response đã chuẩn hóa cho ứng dụng.

Fail-closed fallback của SDK

Nếu request thất bại hoặc response không hợp lệ, SDK trả kết quả tắt an toàn để ứng dụng không bật nhầm tính năng.

Xử lý timeout và lỗi mạng

SDK giới hạn thời gian chờ và chuyển lỗi mạng thành response fallback thay vì làm vỡ giao diện demo.

3.4.3 Tích hợp SDK với Demo Application

Demo Application gọi SDK tại các điểm cần quyết định hiển thị tính năng và cập nhật UI theo kết quả nhận được.

Tạo evaluation context

Context được tạo từ người dùng mẫu đang chọn, bao gồm targeting key, user ID, role và attributes minh họa.

Gọi Evaluation API

SDK gửi request đến backend; Demo Application chỉ nhận kết quả đã chuẩn hóa.

Hiển thị hoặc ẩn tính năng

Các component đọc trường `enabled` để hiển thị tính năng mới hoặc giữ trải nghiệm mặc định.

Làm mới trạng thái sau thay đổi cấu hình

Người dùng có thể refresh evaluation để thấy tác động của cấu hình vừa thay đổi trên Dashboard.

3.4.4 User context trong Demo Application

User context được giữ ở mức mẫu, đủ để minh họa targeting nhưng không chứa dữ liệu cá nhân thật.

Targeting key

Targeting key là định danh ổn định dùng cho percentage rollout.

User ID

User ID mẫu phục vụ allowlist và các kịch bản kiểm thử trực quan.

Role

Role mẫu phục vụ rule targeting theo nhóm người dùng.

Organization trong attributes

Organization ID nằm trong attributes để minh họa hướng mở rộng theo tenant hoặc khách hàng doanh nghiệp.

Thuộc tính mở rộng

Các attributes khác có thể được bổ sung mà không làm thay đổi contract chính của SDK.

3.4.5 Hành vi fallback

Demo Application thể hiện rõ nguyên tắc an toàn: khi không chắc chắn, tính năng mới không được bật.

Evaluation API không khả dụng

Khi backend không phản hồi, SDK trả fallback disabled và giao diện hiển thị trạng thái lỗi thân thiện.

Request timeout

Khi vượt quá timeout, request bị hủy ở phía SDK và ứng dụng tiếp tục chạy với kết quả tắt.

Flag không tồn tại

Nếu backend trả `NOT_FOUND`, demo hiển thị tính năng ở trạng thái tắt và reason tương ứng.

Context không đầy đủ

Nếu context thiếu dữ liệu cho một rule, engine bỏ qua rule không đủ điều kiện hoặc trả reason phù hợp; demo dùng reason này để giải thích trạng thái cho người xem.

3.5 Các kịch bản minh họa

Các kịch bản dưới đây dùng để trình diễn sản phẩm, không lặp lại phần giải thích lý thuyết về targeting, rollout, kill switch hay audit log đã trình bày ở Chương 2. Mỗi kịch bản gồm thao tác trên Dashboard, context từ Demo Application và kết quả quan sát được.

Bảng 3.2: Bảng kịch bản minh họa end-to-end của hệ thống

Kịch bản	**Cấu hình/demo data**	**Context hoặc thao tác kiểm chứng**	**Kết quả mong đợi**
Bật toàn cục	Flag `beta-dashboard` hoặc `shipping-progress-meter` có `servingMode=GLOBAL_ON`.	Gọi Evaluation API bằng bất kỳ sample user hợp lệ trong environment production.	`enabled=true`, reason `GLOBAL_ON`.
Targeting theo user	Rule `USER_ALLOWLIST` cho user `demo-user-admin`.	So sánh `demo-user-admin` với user không nằm trong allowlist.	User được allowlist nhận reason `USER_ALLOWLIST`; user khác tiếp tục xét rule thấp hơn hoặc default off.
Targeting theo role	Các flag như `express-payment`, `coupon-engine`, `personalized-recommendations` dùng role `beta-customer` hoặc `shop-admin`.	Chuyển giữa `demo-user-beta`, `demo-user-admin` và `demo-user-regular`.	Beta/admin được bật theo reason `ROLE_MATCH`; regular không match role.
Rollout phần trăm	`coupon-engine` và `personalized-recommendations` rollout 50%; `holiday-promo-banner` 75%; `live-support-widget` 25%.	Thử nhiều `targetingKey` mẫu và tăng/giảm tỷ lệ rollout.	Phân bổ ổn định theo stable hashing, reason `PERCENTAGE_ROLLOUT` khi bucket nằm trong ngưỡng.
Kill switch cấp flag/group	Flag kill switch hoặc group kill switch của `checkout-experience/recommendations`.	Bật kill switch trên Dashboard rồi gọi lại Demo App/Evaluation API.	Tính năng bị tắt ngay, reason `KILL_SWITCH` hoặc `GROUP_KILL_SWITCH`.
Audit, history, statistics	Mutation cấu hình và các request evaluation phát sinh từ Dashboard/Demo App.	Mở màn hình audit log, flag history và statistics sau khi thao tác.	Có actor, action, request ID, before/after snapshot và số liệu aggregate được cập nhật.
SDK fallback	SDK `@ffp/js-sdk` cấu hình timeout/fetch lỗi.	Mô phỏng Evaluation API timeout hoặc không khả dụng.	SDK trả `enabled=false`, `variant=off`, reason `ERROR`, `errorSource=CLIENT`.

3.5.1 Bật và tắt tính năng toàn cục

Người quản trị chọn một Feature Flag ở môi trường production và chuyển serving mode giữa `GLOBAL_ON` và trạng thái không phục vụ toàn cục. Demo Application gọi Evaluation API với cùng context trước và sau thao tác. Kết quả quan sát được là giao diện tính năng mới xuất hiện hoặc biến mất ngay sau khi cấu hình thay đổi, đồng thời audit log ghi nhận mutation tương ứng.

3.5.2 Targeting theo user và role

Với user allowlist, người quản trị thêm một `userId` cụ thể vào rule. Demo Application chọn lần lượt người dùng nằm trong và ngoài danh sách để chứng minh chỉ user được cấu hình nhận `enabled = true`.

Với role targeting, Dashboard cấu hình role được phép, ví dụ `beta-customer`. Demo Application đổi giữa người dùng beta và người dùng thường để quan sát hai kết quả khác nhau. Kịch bản này chứng minh cùng một flag có thể được phát hành cho nhóm người dùng có đặc điểm nghiệp vụ khác nhau mà không cần đổi mã ứng dụng.

3.5.3 Rollout theo phần trăm

Người quản trị đặt rollout ở một tỷ lệ cụ thể, sau đó Demo Application thử nhiều sample user có `targetingKey` khác nhau. Một số người dùng nhận tính năng mới, số khác tiếp tục ở trạng thái cũ. Khi tỷ lệ rollout tăng, nhóm đã được bật không bị đảo trạng thái; khi tỷ lệ giảm, chỉ các bucket nằm ngoài ngưỡng mới bị tắt.

3.5.4 Kill switch ở cấp flag và group

Khi một flag đang được bật bằng global on hoặc targeting rule, người quản trị kích hoạt flag kill switch. Demo Application nhận ngay kết quả tắt, cho thấy hệ thống có thể rollback runtime mà không cần deploy lại.

Với group kill switch, nhiều flag thuộc cùng một nhóm bị tắt đồng thời trong environment hiện tại. Kịch bản này dùng để trình bày khả năng phản ứng nhanh khi một nhóm tính năng liên quan gây lỗi hoặc cần tạm dừng trong production.

3.5.5 Audit log, history và statistics

Sau mỗi thao tác cấu hình, Dashboard hiển thị bản ghi audit mới với actor, action, request ID và snapshot trước/sau. Màn hình history dùng cùng nguồn dữ liệu này để truy vết thay đổi theo từng flag.

Khi Demo Application phát sinh nhiều request evaluation, màn hình statistics cập nhật tổng số lượt đánh giá và phân bố bật/tắt. Đây là bằng chứng cho thấy hệ thống không chỉ thay đổi hành vi runtime mà còn cung cấp dữ liệu quan sát phục vụ quản trị.

3.5.6 Fallback của SDK

Kịch bản cuối cùng mô phỏng Evaluation API không khả dụng hoặc timeout. JavaScript SDK trả về giá trị fallback tắt, giúp Demo Application tiếp tục hoạt động ở trạng thái an toàn. Kết quả này minh họa nguyên tắc fail-closed trong luồng tích hợp ứng dụng khách.

3.6 Kết quả kiểm thử

Phần này ghi nhận kết quả sau khi áp dụng chiến lược kiểm thử đã nêu ở Chương 2. Nội dung tập trung vào bằng chứng đạt/không đạt, không lặp lại chi tiết thiết kế của engine, API, cache, RBAC hoặc Docker Compose.

3.6.1 Tổng quan kết quả

Các nhóm kiểm thử chính đều hoàn thành với kết quả đạt. Rule Evaluation Engine trả về đúng precedence và reason code; Backend API giữ contract response ổn định; mutation tạo audit log và invalidation cache như kỳ vọng; Dashboard phản ánh đúng quyền thao tác; Demo Application nhận kết quả từ SDK và thay đổi giao diện theo trạng thái flag.

Bảng 3.3: Tổng hợp kết quả kiểm thử tự động sau khi chạy `npm run test - -runInBand`

Nhóm kiểm thử	Minh chứng trong source code	Số lượng	Kết quả
Backend API, persistence, audit, RBAC, cache, statistics và evaluation engine	`apps/backend/src/**/*.spec.ts`	52 test suites, 416 tests	416/416 passed
JavaScript SDK fail-closed client	`packages/js-sdk/test/client.spec.ts`	1 test suite, 21 tests	21/21 passed
Tổng toàn workspace	Root command `npm run test - -runInBand`	53 test suites, 437 tests	437/437 passed

3.6.2 Rule Evaluation Engine

Unit test xác nhận các nhánh quyết định quan trọng: archive, disabled, group kill switch, flag kill switch, global on, user allowlist, role targeting, percentage rollout, default off, `NOT_FOUND`, `INVALID_CONTEXT` và `ERROR`. Kết quả cho thấy engine luôn dừng ở điều kiện có precedence cao nhất và không đánh giá các rule thấp hơn khi đã có kết quả terminal.

Đối với percentage rollout, cùng một `targetingKey` luôn được ánh xạ tới cùng một bucket khi cấu hình không đổi. Khi tăng tỷ lệ rollout, tập người dùng đã được bật tiếp tục được giữ lại;

khi giảm tỷ lệ, chỉ các bucket nằm ngoài ngưỡng mới bị tắt.

3.6.3 Backend API và persistence

Integration test xác nhận các API quản trị xử lý đúng dữ liệu hợp lệ, từ chối dữ liệu sai định dạng và phân biệt được các tình huống `NOT_FOUND`, conflict và validation error. Evaluation API luôn trả response an toàn với `enabled = false` cho các trường hợp không thể đánh giá.

Các mutation quan trọng đều tạo audit log append-only với actor, action, request ID và snapshot trước/sau. Các kiểm thử cũng xác nhận dữ liệu audit không bị sửa hoặc xóa thông qua API quản trị thông thường.

3.6.4 Cache, statistics và RBAC

Cache test cho thấy request đầu tiên tải snapshot từ cơ sở dữ liệu, các request sau dùng cache, và cache liên quan được invalidation sau mutation. Khi cache lỗi, Evaluation API fallback về repository và vẫn giữ kết quả fail-closed đúng contract.

Statistics test xác nhận số lần evaluation được tổng hợp theo flag, kết quả bật/tắt và mốc thời gian phục vụ Dashboard. RBAC test xác nhận ba vai trò demo hoạt động đúng: admin có toàn quyền, developer chỉ được phép trong phạm vi cấu hình được cấp, viewer chỉ đọc. Các thao tác vượt quyền bị chặn ở backend và được phản ánh trên Dashboard.

3.6.5 Dashboard, Demo Application và Docker Compose

Kiểm thử giao diện xác nhận Dashboard hiển thị đúng danh sách project/flag/rule/group, trạng thái lifecycle/runtime, audit log, statistics và trạng thái vô hiệu hóa theo role. Demo Application gọi Evaluation API thông qua JavaScript SDK, hiển thị tính năng theo kết quả `enabled` và dùng fallback tắt khi API không khả dụng.

Quy trình Docker Compose build và khởi chạy thành công các thành phần chính gồm PostgreSQL, Backend API, Admin Dashboard và Demo Application. Sau migration và seed, hệ thống có dữ liệu demo đủ để trình bày các kịch bản end-to-end mà không cần cấu hình thủ công ban đầu.

3.7 Đánh giá kết quả thực hiện

Phần này đánh giá mức độ hoàn thành so với yêu cầu mini project và giá trị của sản phẩm. Các mô tả chi tiết về từng API, màn hình, test case và kịch bản demo đã được trình bày ở các section trước nên không lặp lại tại đây.

3.7.1 Mức độ đáp ứng yêu cầu

Hệ thống đáp ứng nhóm yêu cầu bắt buộc: có báo cáo nghiên cứu, Backend API, Admin Dashboard, Demo Application, dữ liệu lưu trữ bền vững, validation, xử lý lỗi, seed data, tài liệu thiết kế và kịch bản trình diễn. Các chức năng cốt lõi của Feature Flag gồm quản lý flag, cấu hình rule, evaluation runtime và audit log đều hoạt động trong luồng end-to-end.

Ngoài MVP, hệ thống đã hoàn thành nhiều yêu cầu khuyến khích: cache evaluation snapshot, JavaScript SDK, unit/integration test cho rule engine và API, history dựa trên audit log, RBAC demo, statistics, group kill switch và Docker Compose.

3.7.2 Đánh giá thiết kế kỹ thuật

Điểm mạnh chính của thiết kế là tách rõ control plane và data plane. Control plane xử lý cấu hình, phân quyền và audit; data plane tập trung vào evaluation nhanh, xác định và fail-closed. Sự phân tách này giúp giải thích kiến trúc rõ ràng trong báo cáo và giảm rủi ro khi mở rộng runtime evaluation sau này.

Rule Evaluation Engine có precedence tường minh và reason code ổn định, giúp kết quả dễ kiểm thử. Mô hình dữ liệu tách định nghĩa flag khỏi cấu hình theo environment, hỗ trợ group kill switch và lưu audit log append-only. Dashboard và SDK bám theo contract của backend nên các thành phần phối hợp nhất quán.

3.7.3 Đánh giá giá trị thực tế

Sản phẩm chứng minh được giá trị thực tế của Feature Flag trong ba tình huống: phát hành tính năng theo phạm vi nhỏ trước khi mở rộng, rollback nhanh bằng kill switch và quan sát lịch sử thay đổi khi cấu hình runtime làm thay đổi hành vi ứng dụng. Demo app giúp người xem thấy trực tiếp sự khác biệt giữa deploy mã nguồn và release tính năng.

So với LaunchDarkly hoặc Unleash, mini project không hướng tới đầy đủ chức năng thương

mại. Giá trị của hệ thống nằm ở việc tái hiện các nguyên lý cốt lõi bằng một kiến trúc nhỏ, dễ giải thích và có đủ bằng chứng triển khai.

3.7.4 Hạn chế còn lại

Hệ thống hiện vẫn là một bản mini project. Các hạn chế chính gồm: chưa đánh giá ở lưu lượng lớn, chưa có đồng bộ cấu hình realtime giữa nhiều instance, số loại rule còn giới hạn, SDK mới có JavaScript, mô hình xác thực chỉ phục vụ demo, và chưa có monitoring/backup/recovery ở mức production.

Những hạn chế này không làm thay đổi kết quả MVP nhưng xác định rõ các hướng phát triển cần ưu tiên nếu hệ thống tiếp tục được mở rộng sau chương trình.

Chương 4

Kết luận

4.1 Tổng kết kết quả đạt được

Dề tài đã hoàn thành mục tiêu nghiên cứu Feature Flag và xây dựng một nền tảng quản lý phát hành tính năng an toàn ở phạm vi mini project. Báo cáo đã trình bày nền tảng lý thuyết, khảo sát giải pháp hiện có, phân tích yêu cầu, thiết kế kiến trúc, thiết kế dữ liệu, Rule Evaluation Engine, API, bảo mật, kiểm thử và kết quả triển khai.

Về sản phẩm, hệ thống gồm Backend API, Admin Dashboard, Demo Application, JavaScript SDK và PostgreSQL. Các chức năng bắt buộc được triển khai đầy đủ; nhiều chức năng khuyến khích như cache snapshot, audit-backed history, RBAC demo, statistics, group kill switch và Docker Compose cũng được hoàn thành.

Về kiểm chứng, hệ thống đã được kiểm thử ở các lớp engine, API, persistence, cache, RBAC, giao diện, demo app và quy trình triển khai. Các kịch bản trình diễn cho thấy người quản trị có thể thay đổi cấu hình flag trên Dashboard và quan sát hành vi ứng dụng thay đổi thông qua Evaluation API mà không cần deploy lại.

Như vậy, kết quả đạt được không chỉ là một ứng dụng quản lý bật/tắt tính năng, mà là một mô hình nhỏ thể hiện được tư duy release management, thiết kế hệ thống, phân tách control plane/data plane, fail-closed evaluation và auditability trong vận hành phần mềm.

4.2 Các đóng góp chính

Các đóng góp của đề tài được tổng hợp theo ba nhóm, tránh lặp lại phần mô tả chi tiết đã trình bày ở các chương trước.

4.2.1 Đóng góp về nghiên cứu

Báo cáo hệ thống hóa kiến thức về Feature Flag trong bối cảnh release management: tách deploy khỏi release, phân loại flag, các chiến lược targeting/rollout, rủi ro feature flag debt và yêu cầu vận hành an toàn. Việc khảo sát LaunchDarkly, Unleash, OpenFeature và các khuyến nghị bảo mật giúp xác định những thành phần cốt lõi cần có trong một nền tảng Feature Flag thu nhỏ.

4.2.2 Đóng góp về thiết kế

Đề tài đề xuất thiết kế có ranh giới rõ giữa control plane và data plane. Control plane đảm nhiệm quản trị cấu hình, phân quyền và audit log; data plane đảm nhiệm evaluation với snapshot, precedence, reason code và fallback an toàn. Mô hình dữ liệu hỗ trợ project, environment, flag, config, rule, group, sample context, audit log và statistics theo cách đủ đơn giản cho mini project nhưng vẫn có khả năng mở rộng.

4.2.3 Đóng góp về triển khai

Sản phẩm hiện thực đầy đủ luồng end-to-end từ quản trị flag đến thay đổi hành vi trong demo app. Backend, Dashboard, SDK, Demo Application, migration/seed, test và Docker Compose tạo thành một hệ thống có thể chạy, kiểm thử và trình bày được. Đây là bằng chứng thực tế cho các quyết định thiết kế đã phân tích trong báo cáo.

4.2.4 Đóng góp về tư duy hệ thống

Đề tài thể hiện cách tiếp cận hệ thống đối với một bài toán tưởng như chỉ là bật/tắt tính năng. Mỗi quyết định đều được đặt trong quan hệ giữa độ an toàn, khả năng giải thích, hiệu năng, bảo mật, trải nghiệm quản trị và khả năng mở rộng. Đây là giá trị quan trọng nhất của mini project khi so sánh với việc chỉ xây dựng một API CRUD đơn giản.

4.3 Bài học kinh nghiệm

Quá trình thực hiện đề tài cho thấy giá trị của việc tách rõ nghiên cứu, thiết kế, triển khai và kiểm thử. Các bài học dưới đây được trình bày ở mức tổng kết, không lặp lại chi tiết kỹ thuật đã phân tích ở các chương trước.

4.3.1 Bài học về phân tích bài toán

Xác định phạm vi MVP

Feature Flag là miền bài toán rộng, dễ mở sang A/B testing, realtime sync hoặc workflow phê duyệt. Việc xác định MVP giúp ưu tiên các năng lực cốt lõi: quản lý flag, rule engine xác định, audit log, dashboard và demo tích hợp.

Phân tách yêu cầu bắt buộc và mở rộng

Tách yêu cầu bắt buộc khỏi yêu cầu khuyến khích giúp sản phẩm có baseline ổn định trước khi bổ sung SDK, RBAC, statistics, group kill switch và Docker Compose.

Chuyển yêu cầu thành use case và thiết kế

Các yêu cầu ban đầu cần được chuyển thành use case, contract API, mô hình dữ liệu và tiêu chí kiểm thử cụ thể; nếu không, hệ thống dễ chỉ dừng ở giao diện bật/tắt đơn giản.

4.3.2 Bài học về thiết kế hệ thống

Tầm quan trọng của evaluation precedence

Một rule engine chỉ đáng tin cậy khi thứ tự ưu tiên được định nghĩa trước và được kiểm thử. Dây là điểm quyết định để kết quả evaluation có thể giải thích được.

Phân tách định nghĩa flag và cấu hình runtime

Tách metadata của flag khỏi cấu hình theo environment giúp cùng một flag phục vụ nhiều môi trường mà không nhân bản định nghĩa.

Thiết kế auditability từ đầu

Audit log nên được đưa vào luồng mutation ngay từ đầu, vì bổ sung sau khi hệ thống đã có nhiều endpoint sẽ dễ bỏ sót hành động quan trọng.

Cân bằng giữa tính đơn giản và khả năng mở rộng

Kiến trúc một backend service đủ đơn giản cho mini project nhưng vẫn giữ ranh giới module để có thể tách service hoặc thêm hạ tầng phân tán sau này.

4.3.3 Bài học về triển khai và kiểm thử

Validation tại ranh giới hệ thống

DTO validation và constraint database giúp phát hiện lỗi sớm, giảm trạng thái cấu hình không hợp lệ lọt vào engine.

Kiểm thử các trường hợp precedence

Các test quan trọng nhất là những test chứng minh precedence, fail-closed và stable rollout không thay đổi khi refactor.

Quản lý cache invalidation

Cache chỉ an toàn khi có chiến lược invalidation rõ. Với Feature Flag, snapshot cũ có thể trực tiếp làm sai quyết định runtime.

Xây dựng dữ liệu và kịch bản demo

Seed data và kịch bản demo cần được chuẩn bị cùng lúc với tính năng để người xem có thể quan sát giá trị hệ thống mà không phải cấu hình thủ công quá nhiều.

4.4 Hạn chế

Hệ thống đã đáp ứng mục tiêu mini project nhưng vẫn chưa phải một nền tảng Feature Flag thương mại hoàn chỉnh. Các hạn chế được nhóm lại theo kiến trúc, chức năng và vận hành.

4.4.1 Hạn chế về kiến trúc

Chưa hỗ trợ triển khai phân tán

Backend hiện phù hợp với demo một cụm nhỏ; khi chạy nhiều instance, hệ thống cần cơ chế đồng bộ cache và cấu hình tốt hơn.

Chưa có cơ chế đồng bộ cấu hình realtime

SDK hiện gọi API theo yêu cầu, chưa nhận push update qua WebSocket, SSE hoặc streaming.

Chưa đánh giá ở lưu lượng lớn

Các kiểm thử hiện tập trung vào tính đúng đắn và luồng demo, chưa có benchmark tải lớn cho Evaluation API.

4.4.2 Hạn chế về chức năng

Số loại rule còn giới hạn

Engine mới hỗ trợ allowlist, role targeting và percentage rollout; các điều kiện theo thuộc tính, thời gian hoặc vị trí chưa được triển khai.

Chưa hỗ trợ nhiều variant đầy đủ

Response đã có chỗ cho variant nhưng chưa có mô hình phân phối đa biến thể và đo lường A/B testing hoàn chỉnh.

Chưa hỗ trợ approval workflow

Mutation cấu hình được áp dụng trực tiếp sau khi có quyền, chưa có bước đề xuất/phê duyệt cho production.

Chưa hỗ trợ stale flag detection

Hệ thống chưa tự phát hiện flag không còn được sử dụng hoặc đã tồn tại quá lâu.

4.4.3 Hạn chế về bảo mật và vận hành

Mô hình xác thực còn đơn giản

RBAC hiện là mô hình demo, chưa tích hợp OAuth2/OIDC, SSO hoặc quản lý phiên đầy đủ.

Chưa có rate limiting đầy đủ

Evaluation API và Management API chưa có quota/rate limit theo actor, IP hoặc project.

Chưa có hệ thống monitoring chuyên biệt

Statistics hiện phục vụ nghiệp vụ Feature Flag, chưa thay thế monitoring hạ tầng như latency, error rate, CPU hoặc memory.

Chưa triển khai cơ chế backup và recovery

Quy trình sao lưu, khôi phục và diễn tập disaster recovery chưa nằm trong phạm vi hiện tại.

4.5 Hướng phát triển

Các hướng phát triển dưới đây trực tiếp xuất phát từ hạn chế đã nêu, nhằm tránh lặp lại phần đánh giá kết quả.

4.6 Mở rộng Rule Evaluation Engine

4.6.1 Targeting theo thuộc tính

Bổ sung điều kiện theo device, plan, app version, country hoặc thuộc tính nghiệp vụ trong Evaluation Context.

4.6.2 Targeting theo organization

Hỗ trợ bật/tắt tính năng theo tenant hoặc khách hàng doanh nghiệp, phù hợp với hệ thống SaaS.

4.6.3 Targeting theo thời gian và vị trí

Cho phép phát hành theo lịch, chiến dịch hoặc khu vực địa lý.

4.6.4 Hỗ trợ nhiều variant và A/B testing

Mở rộng response và mô hình dữ liệu để phân phối nhiều biến thể, kết hợp metrics để đánh giá hiệu quả thử nghiệm.

4.7 Mở rộng khả năng tích hợp

4.7.1 SDK cho nhiều ngôn ngữ

Phát triển SDK cho Java, Go, Python, C# hoặc Kotlin với contract nhất quán.

4.7.2 Webhook khi cấu hình thay đổi

Gửi sự kiện đến hệ thống ngoài khi flag, rule hoặc group thay đổi.

4.7.3 Đồng bộ cấu hình theo thời gian thực

Sử dụng WebSocket, Server-Sent Events hoặc streaming để SDK nhận cấu hình mới nhanh hơn polling.

4.7.4 Tương thích OpenFeature

Điều chỉnh SDK/API theo các khái niệm provider, evaluation context và resolution details của OpenFeature [14, 13].

4.8 Mở rộng quản trị vòng đời flag

4.8.1 Approval workflow

Thêm quy trình đề xuất và phê duyệt trước các thay đổi production nhạy cảm.

4.8.2 Lập lịch bật và tắt flag

Cho phép cấu hình thời điểm tự động thay đổi trạng thái flag.

4.8.3 Phát hiện stale flag

Dựa trên thời gian không cập nhật, không được evaluate hoặc luôn cho cùng kết quả để đề xuất dọn dẹp.

4.8.4 Nhắc nhở và tự động dọn dẹp flag

Tự động nhắc owner hoặc archive flag theo chính sách đã cấu hình.

4.9 Mở rộng khả năng vận hành

4.9.1 High availability

Triển khai nhiều backend instance, load balancer và database replication để tăng độ sẵn sàng.

4.9.2 Distributed cache

Dùng Redis hoặc cơ chế tương đương để chia sẻ cache/invalidation giữa nhiều instance.

4.9.3 Monitoring và alerting

Thu thập latency, error rate, cache hit/miss và tài nguyên hệ thống, sau đó cảnh báo khi vượt ngưỡng.

4.9.4 Triển khai trên Kubernetes

Đóng gói workload cho Kubernetes để hỗ trợ scaling, rolling update và quản lý cấu hình ở môi trường production.

Tài liệu tham khảo

[1] Pete Hodgson. *Feature Toggles (aka Feature Flags)*. Martin Fowler. 9 **october** 2017. URL: https://martinfowler.com/articles/feature-toggles.html.

[2] Unleash. *Feature Flags*. Unleash Documentation. URL: https://docs.getunleash.io/concepts/feature-flags.

[3] LaunchDarkly. *Feature Flags*. LaunchDarkly Documentation. URL: https://docs.launchdarkly.com/home/flags.

[4] Microsoft. *How Microsoft Develops with DevOps*. Microsoft Learn. 28 **november** 2022. URL: https://learn.microsoft.com/en-us/devops/develop/how-microsoft-develops-devops.

[5] Unleash. *11 Best Practices for Building and Scaling Feature Flag Systems*. Unleash Documentation. URL: https://docs.getunleash.io/topics/feature-flags/feature-flag-best-practices.

[6] LaunchDarkly. *Targeting Rules*. LaunchDarkly Documentation. URL: https://launchdarkly.com/docs/home/flags/target-rules.

[7] LaunchDarkly. *Percentage Rollouts*. LaunchDarkly Documentation. URL: https://launchdarkly.com/docs/home/releases/percentage-rollouts.

[8] Rezvan Mahdavi-Hezaveh, Jacob Dremann **and** Laurie Williams. **?**Software Development with Feature Toggles: Practices Used by Practitioners**? in***Empirical Software Engineering*: 26, 1 (8 **january** 2021). DOI: 10.1007/s10664-020-09901-z. URL: https://link.springer.com/article/10.1007/s10664-020-09901-z.

[9] OWASP Foundation. *REST Security Cheat Sheet*. OWASP Cheat Sheet Series. URL: https://cheatsheetseries.owasp.org/cheatsheets/REST_Security_Cheat_Sheet.html.

[10] OWASP Foundation. *Logging Cheat Sheet*. OWASP Cheat Sheet Series. URL: https://cheatsheetseries.owasp.org/cheatsheets/Logging_Cheat_Sheet.html.

[11] PostgreSQL Global Development Group. *JSON Types*. PostgreSQL Documentation. URL: https://www.postgresql.org/docs/current/datatype-json.html.

[12] Docker. *Docker Compose*. Docker Documentation. URL: https://docs.docker.com/compose/.

[13] OpenFeature. *Flag Evaluation API*. OpenFeature Specification. URL: https://openfeature.dev/specification/sections/flag-evaluation/.

[14] OpenFeature. *Evaluation Context*. OpenFeature Specification. URL: https://openfeature.dev/specification/sections/evaluation-context/.